

第一章：

一、名词解释

中央处理器（CPU）：中央处理器是计算机系统的核心部件，负责执行程序中的指令、处理数据，并控制计算机的各个部件协调工作。CPU 包含算术逻辑单元（ALU）和控制单元，执行算术运算、逻辑运算、数据传送和输入/输出操作的控制。

算术逻辑部件（ALU）：算术逻辑部件是 CPU 中的一个关键组件，负责执行算术运算（如加、减、乘、除）和逻辑运算（如与、或、非、异或）。ALU 接收来自控制单元的操作码，并根据这些操作码执行相应的运算。

通用寄存器：通用寄存器是 CPU 内部的高速存储单元，用于暂存指令、数据和地址。它们可以被 CPU 快速访问，从而加速数据处理速度。

程序计数器（PC）：程序计数器是一个特殊的寄存器，用于存储下一条将要执行的指令的地址。当 CPU 执行一条指令时，程序计数器自动加 1（或按照跳转指令更改），以指向下一条要执行的指令。

指令寄存器（IR）：指令寄存器用于暂存当前正在执行的指令。当 CPU 从主存储器中读取一条指令后，会将其放入指令寄存器中，解码并执行。

控制器：控制器是 CPU 中的另一个关键组件，负责从主存储器中取出指令，对指令进行译码，并根据指令的要求，有序地、协调地指挥和控制计算机各部件自动工作。

主存储器：主存储器是计算机中的主要存储部件，用于存储程序和数据。CPU 可以直接访问主存储器中的数据，而不需要通过其他外部设备。

总线：总线是计算机各部件之间传输信息的公共通道，由导线组成，用于在 CPU、主存储器、I/O 设备之间传输数据和控制信号。

主存地址寄存器（MAR）：主存地址寄存器用于存放欲访问的主存单元的地址，其地址由 CPU 中的程序计数器给出。MAR 与主存储器地址译码电路相连，为 CPU 访问主存提供所要访问的主存单元的地址。

主存数据寄存器（MDR）：主存数据寄存器用于暂存从主存储器读出的一条指令或一个数据字，或准备写入主存储器的字。MDR 与主存储器数据总线相连，与 MAR 配合使用，实现 CPU 与主存储器之间的数据交换。

机器指令：机器指令是 CPU 能直接识别并执行的指令，由操作码和操作数地址码组成。每一条指令完成一个基本的微操作，如数据传送、算术运算等。

指令操作码：指令操作码是机器指令中的一部分，用于指定该指令所要执行的操作类型，

如加法、减法、存储等。

高级程序设计语言：高级程序设计语言是一种接近于人类自然语言和数学语言的程序设计语言，易于编写和理解，经过编译或解释后能转换成机器语言执行。

汇编语言：汇编语言是一种用助记符表示的指令系统中的指令，是一种低级程序设计语言。汇编语言需要汇编程序翻译成机器语言才能执行。

机器语言：机器语言是直接用二进制代码指令表达的计算机语言，是计算机唯一能直接识别和执行的语言。

机器级语言：机器级语言通常指的是与特定机器或处理器直接相关的低级编程语言，包括机器语言和汇编语言。

源程序：源程序是用高级程序设计语言编写的、尚未编译或解释的程序。

目标程序：目标程序是源程序经编译或解释后得到的机器语言程序，可以直接被计算机执行。

编译程序：编译程序是一种将高级程序设计语言编写的源程序翻译成目标程序的软件工具。编译过程通常包括词法分析、语法分析、语义分析、优化和代码生成等阶段。

解释程序：解释程序是一种直接执行高级程序设计语言编写的源程序的软件工具。解释程序对源程序进行逐句扫描和解释，并立即执行相应的操作，不生成独立的目标程序。

汇编程序：汇编程序是一种将汇编语言源程序翻译成机器语言目标程序的软件工具。汇编过程包括将汇编语言指令转换为机器语言指令，并分配内存地址等。

语言处理系统：语言处理系统是指用于处理各种程序设计语言的软件工具集合，包括编译器、解释器、汇编器等。

设备控制器：设备控制器是计算机中的一个硬件部件，负责控制和管理外部设备（如打印机、磁盘驱动器等）与 CPU 之间的数据传输。

最终用户：最终用户是指直接使用计算机系统和应用程序的个人或组织，他们通过用户界面与计算机进行交互，完成各种任务。

系统管理员：系统管理员是负责管理和维护计算机系统的专业人员，他们负责安装、配置、监控和维护计算机硬件、软件和网络等。

应用程序员：应用程序员是负责开发、编写和维护各种应用程序的专业人员，他们使用高级程序设计语言进行程序设计，以满足用户的特定需求。

系统程序员：系统程序员是负责设计、开发和维护计算机系统软件的专业人员，他们负责编写操作系统、编译器、数据库管理系统等核心软件。

指令集体系结构 (ISA): 指令集体系结构定义了计算机指令的格式、数量和功能, 以及 CPU 如何执行这些指令

微体系结构 (Microarchitecture): 微体系结构是计算机处理器设计的一个关键部分, 它定义了处理器内部各个组件的结构、功能以及它们之间的交互方式。微体系结构决定了处理器如何执行指令、管理数据、处理中断等, 从而影响了处理器的性能、功耗和效率。

透明 (Transparency): 在计算机科学中, 透明通常指的是某个方面对使用者是不可见的或者是不需要考虑的。例如, 内存管理的某些细节对程序员可能是透明的, 意味着程序员不需要直接处理这些细节, 而由操作系统或硬件自动管理。

响应时间 (Response Time): 响应时间是指系统或组件从接收到输入或请求到产生相应输出或响应所需的时间。在计算机系统中, 响应时间通常用于衡量用户界面、网络请求或数据库查询等的速度。

吞吐率 (Throughput): 吞吐率是指系统或组件在单位时间内处理输入或输出数据的数量或速度。它衡量了系统处理数据的整体效率, 通常以每秒处理的事务数、每秒传输的字节数等来度量。

用户 CPU 时间 (User CPU Time): 用户 CPU 时间是指 CPU 在执行用户程序时所花费的时间。这是区分于系统 CPU 时间的一个概念, 系统 CPU 时间指的是 CPU 在执行操作系统内核任务时所花费的时间。

时钟周期 (Clock Cycle): 时钟周期是计算机处理器中的一个基本时间单位, 它表示处理器执行一个基本操作所需的时间。每个时钟周期, 处理器都会执行一系列预定义的指令或操作。

主频 (Clock Rate): 主频是指处理器每秒钟的时钟周期数, 通常以赫兹 (Hz) 或兆赫兹 (MHz)、吉赫兹 (GHz) 为单位表示。主频越高, 处理器执行指令的速度通常越快。

CPI (Cycles Per Instruction): CPI 是指处理器执行一条指令所需的平均时钟周期数。它衡量了处理器的指令执行效率, CPI 越低, 说明处理器执行指令的速度越快。

基准程序 (Benchmark Program): 基准程序是一组用于测试和评估计算机系统性能的预定义程序和测试套件。通过运行这些基准程序, 可以比较不同系统的性能, 并找出性能瓶颈。

MIPS (Millions of Instructions Per Second): MIPS 是一个衡量处理器性能的指标, 表示处理器每秒钟能执行的百万条指令数。MIPS 越高, 说明处理器的性能越好。

峰值 MIPS (Peak MIPS): 峰值 MIPS 是指处理器在理想条件下能够达到的最高 MIPS 值。这通常是在处理器没有遇到任何瓶颈 (如内存延迟、I/O 限制等) 时测得的性能上限。

相对 MIPS (Relative MIPS): 相对 MIPS 是用于比较不同处理器或系统性能的 MIPS 指标。它通常是将一个系统的 MIPS 值除以另一个基准系统的 MIPS 值，从而得到一个相对性能的比值。

二、简单回答下列问题

(1)冯·诺依曼计算机由哪几部分组成?各部分的功能是什么?

解答：冯·诺依曼计算机主要由五大部分组成：输入设备、输出设备、运算器、控制器和存储器。

输入设备：用于接收用户输入的数据和指令，如键盘、鼠标等。

输出设备：用于展示计算机处理后的结果，如显示器、打印机等。

运算器：执行各种算术和逻辑运算。

控制器：控制计算机的各个部件协同工作，负责指令的读取、解释和执行。

存储器：存储数据和程序，包括内存和外存。

(2) 什么是“存储程序”工作方式?

解答：“存储程序”工作方式是指将程序和数据以二进制代码形式预先存放在计算机的存储器中，然后计算机在运行时从存储器中取出指令并加以执行。这种方式使得计算机能自动、连续地执行程序，从而大大提高了计算机的工作效率。

(3)一条指令的执行过程包含哪几个阶段?

解答：一条指令的执行过程通常包含以下几个阶段：

取指令：从存储器中取出指令。

分析指令：对指令进行译码，确定其操作性质。

执行指令：根据指令的要求进行具体的操作，如数据的运算、地址的计算、结果的存储等。

存储结果：将执行结果存储回存储器。

(4)计算机系统的层次结构如何划分?

解答：答案：计算机系统的层次结构通常划分为以下几个层次：

微程序机器层：与硬件直接相关的层次，由微指令和微程序组成。

机器语言机器层：由机器指令组成的层次，程序员可以直接用机器指令编写程序。

操作系统机器层：通过系统调用为程序员提供一批汇编、编译、连接和编辑等软件。

汇编语言机器层：为程序员提供汇编语言。

高级语言机器层：使用高级程序设计语言编程的层次，程序员可以通过高级语言编写程序。

(5)计算机系统的用户可分为哪几类?每类用户工作在哪个层次?

解答：计算机系统的用户主要分为三类：

高级用户：这类用户通常工作在应用层，使用高级语言进行编程和应用开发。

系统程序员：他们工作在系统层，负责设计、开发和维护操作系统、编译器等系统软件。

机器语言程序员：这类用户直接工作在机器语言层，使用机器语言或汇编语言编写程序。

(6) 程序的 CPI 与哪些因素有关?

解答：程序的 CPI（每条指令的时钟周期数）与以下因素有关：

指令类型：不同类型的指令所需的执行时间不同，因此 CPI 会有所差异。

指令的执行顺序：指令的执行顺序可能影响到 CPU 的流水线效率，从而影响 CPI。

数据的存取方式：如内存访问延迟、缓存命中率等都会影响指令的执行时间，从而影响 CPI。

程序的优化程度：优化过的程序通常能更有效地利用 CPU 资源，降低 CPI。

(7)为什么说性能指标 MIPS 不能很好地反映计算机的性能?

解答：MIPS（每秒百万条指令数）是一个衡量计算机性能的指标，但它并不能很好地反映计算机的性能，原因如下：

MIPS 只考虑了指令的执行速度，而没有考虑指令的类型和复杂性。不同类型的指令所需的执行时间可能相差很大，因此单纯的指令数量并不能完全反映计算机的性能。

MIPS 忽略了内存访问、I/O 操作等其他可能影响性能的因素。在实际情况中，这些因素往往对性能有着重要影响。

MIPS 是一个理论上的性能指标，实际运行中的程序可能受到多种因素的影响，如操作系统调度、并发执行等，使得实际性能与 MIPS 值存在差距。

因此，虽然 MIPS 是一个有用的性能指标，但它并不能全面反映计算机的实际性能。在评估计算机性能时，还需要考虑其他因素，如响应时间、吞吐量、功耗等。

三、假定你的朋友不太懂计算机,请用简单通俗的语言给你的朋友介绍计算机系统是如何工作的。

解答：计算机系统就像是一个大工厂，有很多不同的工人和机器一起工作来完成任务。

首先，我们有输入设备，比如键盘和鼠标，就像是我们告诉工厂要做什么的手和口。我们通过这些设备告诉电脑我们要做什么。

然后，电脑里有一个叫做“处理器”的东西，它就像工厂里的总指挥，负责安排和执行我们的指令。

接着，电脑还有一个叫做“内存”的地方，就像工厂里的临时仓库，用来存放处理过程中需要用到的数据。

处理器根据我们的指令从内存里取数据，然后进行计算或者操作，再把结果存回内存或者通过输出设备（比如显示器或打印机）展示出来。

最后，电脑里还有一个叫做“硬盘”的地方，就像工厂里的长期仓库，用来保存我们不需要马上用但又不希望丢失的数据。

四、你对计算机系统的哪些部分最熟悉，哪些部分最不熟悉？最想进一步了解细节的是哪些部分的内容？

解答：我最熟悉的部分是操作系统和常用软件的使用，因为平时的工作和生活中经常使用。但对于硬件部分，比如处理器的内部工作原理、内存的层次结构等，我就相对不熟悉。

我最想进一步了解的是计算机网络的细节，比如数据是如何在网络中传输的、网络安全是如何保障的、以及云计算和大数据等技术是如何实现的。这些内容对于现代社会的信息化发展至关重要，而且也是我工作中可能会用到的知识。

五、图 1.1 所示模型计算机(采用图 1.2 所示指令格式)的指令系统中,除了有 $mv(op=000)$ 、 $add(op=00)$ 、 $load(op=110)$ 和 $store(op=111)$ 指令外, R 型指令还有减($sub, op=0010$)和乘($mul, op=0011$)等指令, 请仿照图 1.3 给出求解表达式 " $z=(x-y)*y$;" 所对应的指令序列(包括机器代码和对应的汇编指令)以及主存中的存放内容, 并仿照图 1.5 给出每条指令的执行过程以及所包含的微操作。

解析：首先，我们需要理解提供的指令格式和指令系统。图 1.2 所示指令格式中，操作码（op）占据前 4 位，而 R 型指令的其他位通常用于指定源操作数和目标操作数的寄存器地址。由于未提供具体的指令格式图，我们将基于一般的 R 型指令格式来构建所需的指令序列。

表达式 " $z=(x-y)*y$ " 对应的指令序列需要考虑以下几点：

- （1）*执行减法 $x - y$ ，并将结果保存在一个临时寄存器中（例如 $t0$ ）。
- （2）使用临时寄存器 $t0$ 和 y 执行乘法。
- （3）将乘法结果保存在 z 中。

假设 x 在寄存器 $R1$ 中, y 在寄存器 $R2$ 中, z 在寄存器 $R3$ 中, 而临时结果存放在 $R0$ 中。以下是对应的指令序列和机器代码：

(1) `sub R0, R1, R2` ; $R0 = R1 - R2$

机器代码: 0010 0000 0001 0010 (假设 `op=0010` 为 `sub` 指令, 且寄存器地址按序排列)

汇编指令: `sub R0, R1, R2`

(2) `mul R0, R0, R2` ; $R0 = R0 * R2$ (即 $(x-y) * y$)

机器代码: 0011 0000 0000 0010 (假设 `op=0011` 为 `mul` 指令)

汇编指令: `mul R0, R0, R2`

(3) `mv R3, R0` ; $R3 = R0$ (将结果移动到 `z` 所在的寄存器 `R3`)

机器代码: 0000 0011 0000 0000 (假设 `op=000` 为 `mv` 指令)

汇编指令: `mv R3, R0`

关于在主存中的存放内容, 这取决于程序加载时的内存布局。通常, 指令和数据会分别存放在不同的内存区域。指令序列会被加载到代码段, 而变量 `x`、`y`、`z` 和临时变量 `t0` 则会被加载到数据段。具体的内存地址取决于操作系统的内存管理策略和程序的加载方式。

至于每条指令的执行过程及包含的微操作, 这通常依赖于具体的处理器设计。然而, 我们可以提供一个一般性的描述, 以说明这些指令如何在处理器内部执行:

(1) `sub R0, R1, R2` 指令执行过程:

- 从寄存器 `R1` 和 `R2` 中读取值。
- 执行减法操作。
- 将结果存放到寄存器 `R0`。

(2) `mul R0, R0, R2` 指令执行过程:

- 从寄存器 `R0` 和 `R2` 中读取值。
- 执行乘法操作。
- 将结果存放到寄存器 `R0` (覆盖原来的值)。

(3) `mv R3, R0` 指令执行过程:

- 从寄存器 `R0` 中读取值。
- 将值存放到寄存器 `R3`。

每个指令的执行过程中, 处理器可能需要执行多个微操作, 如从内存读取指令、解码指令、读取源操作数、执行算术运算、写入目标操作数等。这些微操作由处理器的控制单元(CU)根据指令的操作码和指令格式来协调执行。

由于图 1.5 的具体内容未给出, 无法直接仿照其格式来展示每条指令的执行过程和微操作。但 you 可以根据上述描述, 结合你了解的具体处理器架构和微操作, 来绘制每条指令的执

行流程和相应的微操作序列。

六、

6. 若有两个基准测试程序 P1 和 P2 在机器 M1 和 M2 上运行, 假定 M1 和 M2 的价格分别是 5000 元和 8000 元, 下表给出了 P1 和 P2 在 M1 和 M2 上所花的时间和指令条数。

程序	M1		M2	
	指令条数	执行时间	指令条数	执行时间
P1	200×10^6	10 000ms	150×10^6	5000ms
P2	300×10^3	3ms	420×10^3	6ms

请回答下列问题:

- (1) 对于 P1, 哪台机器的速度快? 快多少? 对于 P2 呢?
- (2) 在 M1 上执行 P1 和 P2 的速度分别是多少 MIPS? 在 M2 上的执行速度又各是多少? 从执行速度来看, 对于 P2, 哪台机器的速度快? 快多少?
- (3) 假定 M1 和 M2 的时钟频率各是 800MHz 和 1.2GHz, 则在 M1 和 M2 上执行 P1 时的平均时钟周期数 (CPI) 各是多少?
- (4) 如果某个用户需要大量使用程序 P1, 并且该用户主要关心系统的响应时间而不是吞吐率, 那么, 该用户需要大批购进机器时, 应该选择 M1 还是 M2? 为什么? (提示: 从性价比上考虑。)
- (5) 如果另一个用户也需要购进大批机器, 但该用户使用 P1 和 P2 一样多, 主要关心的也是响应时间, 那么, 应该选择 M1 还是 M2? 为什么? CSDN @call me by ur name

解析:

- (1) 对于 P1, M2 比 M1 快一倍; 对于 P2, M1 比 M2 快一倍。
- (2) 对于 M1, P1 的速度为: $200\text{M}/10=20\text{MIPS}$; P2 为 $300\text{k}/0.003=100\text{MIPS}$ 。对于 M2, P1 的速度为: $150\text{M}/5=30\text{MIPS}$; P2 为 $420\text{k}/0.006=70\text{MIPS}$ 。从执行速度来看, 对于 P2, 因为 $100/70=1.43$ 倍, 所以 M1 比 M2 快 0.43 倍。
- (3) 在 M1 上执行 P1 时的平均时钟周期数 CPI 为: $10 \times 800\text{M}/(200 \times 10^6)=40$ 。在 M2 上执行 P1 时的平均时钟周期数 CPI 为: $5 \times 1.2\text{G}/(150 \times 10^6)=40$ 。
- (4) 考虑运行 P1 时 M1 和 M2 的性价比, 因为该用户主要关心系统的响应时间, 所以性价比中的性能应考虑执行时间, 其性能为执行时间的倒数。故性价比 R 为: $R=1/(\text{执行时间} \times \text{价格})$, R 越大说明性价比越高, 也即, “执行时间 $\times$ 价格” 的值越小, 则性价比越高。因为 $10 \times 5000 > 5 \times 8000$, 所以, M2 的性价比高。应选择 M2。
- (5) P1 和 P2 需要同等考虑, 性能有多种方式: 执行时间总和、算术平均、几何平均。若用算术平均方式, 则: 因为 $(10+0.003)/2 \times 5000 > (5+0.006)/2 \times 8000$, 所以 M2 的性价比高, 应选择 M2。若用几何平均方式, 则: 因为 $\sqrt{10 \times 0.003} \times 5000 < \sqrt{5 \times 0.006} \times 8000$, 所以 M1 的性价比高, 应选择 M1。

7. 若机器 M1 和 M2 具有相同的指令集,其时钟频率分别为 1GHz 和 1.5GHz。在指令集中有 A~E 五种不同类型的指令。下表给出了在 M1 和 M2 上每类指令的平均时钟周期数 CPI。

机器	A	B	C	D	E
M1	1	2	2	3	4
M2	2	2	4	5	6

请回答下列问题:

(1) M1 和 M2 的峰值 MIPS 各是多少?

(2) 假定某程序 P 的指令序列中,五类指令具有完全相同的指令条数,则程序 P 在 M1 和 M2 上运行时,哪台机器更快?快多少?在 M1 和 M2 上执行程序 P 时的平均时钟周期数 CPI 各是多少?

CSDN @call me by ur name

(1)M1. 上可以选择一段都是 A 类指令组成的程序,其峰值 MIPS 为 1000MIPS。M2.上可以选择一段 A 和 B 类指令组成的程序,其峰值 MIPS 为 $1500/2=750$ MIPS。

(2)5 类指令具有完全相同的指令条数,所以各占 20%。在 M1 和 M2 上执行程序 P 时的平均时钟周期数 CPI 分别为:

- $M1:20\% \times (1+2+2+3+4) = 0.2 \times 12 = 2.4$

- $M2:20\% \times (2+2+4+5+6) = 0.2 \times 19 = 3.8$

假设程序 P 的指令条数为 N,则在 M1 和 M2.上的执行时间分别为:

- $M1:2.4 \times N \times 1/1G = 2.4N \text{ (ns)}$

- $M2:3.8 \times N \times 1/1.5G = 2.53 N \text{ (ns)}$

M1 执行 P 的速度更快每条指令平均快 0.13ns,也即 M1 比 M2 快 $0.13/2.53 \times 100\% \approx 5\%$ 。

八. 假设同一套指令集用不同的方法设计了两种机器 M1 和 M2。机器 M1 的时钟周期为 0.8ns, 机器 M2 的时钟周期为 1.2ns。某程序 P 在机器 M1 上运行时的 CPI 为 4,在 M2 上的 CPI 为 2。对于程序 P 来说, 哪台机器的执行速度更快?快多少?

解析: 因为 M1 和 M2 实现的是同一套指令集, 所以程序 P 在机器 M1 和 M2.上执行的指令条数相同,假定是 N 条, 则 P 在 M1 上的执行时间为 $4 \times 0.8ns \times N = 3.2N(ns)$; P 在 M2.上的执行时间为 $2 \times 1.2ns \times N = 2.4N(ns)$ 。由此可知, 对于程序 P 来说, M2 的执行速度更快, 平均每条指令快 0.8ns。

九. 假设某机器 M 的时钟频率为 4CHz,用户程序 P 在 M 上的指令条数为 8×10^6 .其 CPI 为 1.25, 则 P 在 M 上的执行时间是多少?若在机器 M 上从程序 P 开始启动到执行结束所需的时间是 4s, 则 P 的用户 CPU 时间所占的百分比是多少?

解析: 我们要计算用户程序 P 在机器 M 上的执行时间, 以及程序 P 的用户 CPU 时间所占的百分比。

首先，我们需要知道几个关键参数：机器的时钟频率、程序的指令条数、CPI（每条指令的时钟周期数）。

执行时间（T）可以通过以下公式计算：

$$T = (\text{指令条数} \times \text{CPI}) / \text{时钟频率}$$

这里，指令条数是 8×10^6 ，CPI 是 1.25，时钟频率是 4GHz（即 $4 \times 10^9 \text{ Hz}$ ）。

用户 CPU 时间所占的百分比可以通过以下公式计算：

$$\text{百分比} = (\text{P 的执行时间} / \text{总时间}) \times 100\%$$

这里，总时间是 4 秒。

计算结果为：执行时间是 0.0025 秒

用户 CPU 时间所占的百分比是 0.06%。

十、假定某编译器对某段高级语言程序编译生成两种不同的指令序列 S1 和 S2，在时钟频率为 500 MHz 的机器 M 上运行，目标指令序列中用到的指令类型有 A、B、C 和 D 四类。四类指令在 M 上的 CPI 和两个指令序列所用的各类指令条数如下表所示。

	A	B	C	D
各类指令的 CPI	1	2	3	4
S1 的指令条数	5	2	2	1
S2 的指令条数	1	1	1	5

请问 S1 和 S2 各有多少条指令？CPI 各为多少？所含的时钟周期数各为多少？执行时间各为多少？

解析：S1 有 10 条指令，CPI 为 $(5 \times 1 + 2 \times 2 + 2 \times 3 + 1 \times 4) / 10 = 1.9$ ，所含的时钟周期数为 $10 \times 1.9 = 19$ ，执行时间为 $19 / 500\text{M} = 38\text{ns}$ 。

S2 有 8 条指令，CPI 为 $(1 \times 1 + 1 \times 2 + 1 \times 3 + 5 \times 4) / 8 = 3.25$ ，所含的时钟周期数为 $8 \times 3.25 = 26$ ，执行时间为 $26 / 500\text{M} = 52\text{ns}$ 。

(注：从上述结果来看，对于同一个高级语言源程序，在同一台机器上所生成的目标程序不同，其执行时间可能不同，而且，并不是指令条数少的目标程序执行时间就一定少。)

十一、假定机器 M 的时钟频率为 400MHz，某程序 P 在机器 M 上的执行时间为 12.000s。对 P 优化时，将其所有的乘 4 指令都换成了一条左移两位的指令，得到优化后的程序 P'。已知在 M 上乘法指令的 CPI 为 102，左移指令的 CPI 为 2，P' 的执行时间为 1.008。则 P 中有多少条乘法指令被替换成了左移指令被执行？

显然，P 的执行时间为 10 秒，因此，P 比 P' 多花了 2 秒钟，因此，执行时被换成左移指令的

乘法指令的条数为 $1.2\text{GX}2 / (5-) = 800\text{M}$.

第二章：数据的表示和运算

一、给出以下概念的解释说明。

真值：真值是数学上的值，即我们实际想要表示的数，不考虑计算机内部如何表示。

机器数：机器数是在计算机内部用来表示数值的二进制编码。它可能由于编码方式（如原码、补码等）和机器的特性而有所不同。

数值数据：数值数据是指那些可以表示数量或大小的数据，如整数、浮点数等。

非数值数据：非数值数据是指不能表示为数量或大小的数据，如字符、文本、图片等。

无符号整数：无符号整数是一个非负的二进制数，它只表示大小，不表示正负。

带符号整数：带符号整数可以表示正数或负数。在计算机中，通常使用最高位作为符号位，其余位表示数值大小。

定点数：定点数是一种数值表示方法，其中小数点位置是固定的。定点数主要用于表示整数或小数。

浮点数：浮点数是一种数值表示方法，其中小数点位置可以浮动。它通常由尾数（mantissa）和阶码（exponent）两部分组成，用于表示非常大或非常小的数。

尾数：尾数是浮点数中用于表示数值大小的部分。

阶和阶码：阶是浮点数中小数点实际位置与默认位置之间的偏移量。阶码则是这个偏移量的编码表示。

溢出：溢出是指计算机在运算过程中，结果超出了它能表示的范围。这可能导致不正确的结果或错误。

规格化数：规格化数是指满足特定条件的浮点数，例如尾数的最高位是 1（对于二进制数）。这种表示方式可以提高浮点数的精度和范围。

左规：左规是在浮点数运算中，当尾数向左移动时，阶码相应减小的过程。这通常发生在尾数溢出时。

右规：右规是在浮点数运算中，当尾数向右移动时，阶码相应增加的过程。这通常发生在尾数下溢时。

ASCII 码：ASCII（American Standard Code for Information Interchange）码是一种用于表示英文字符和其他控制字符的编码系统。

汉字输入码：汉字输入码是用户通过键盘等设备输入汉字时所使用的编码，如拼音、五

笔等。

汉字内码：汉字内码是计算机内部用于表示汉字的编码，如 GB2312、GBK、UTF-8 等。

字长：字长是指计算机内部处理数据的位数，例如 32 位或 64 位处理器。它决定了计算机一次能处理的数据量。

大端方式：大端方式（Big Endian）是一种数据存储方式，其中高位字节存储在内存的低地址端，低位字节存储在内存的高地址端。

小端方式：小端方式（Little Endian）是另一种数据存储方式，其中低位字节存储在内存的低地址端，高位字节存储在内存的高地址端。

ALU（算术逻辑单元）：ALU 是计算机中的一个重要组件，负责执行算术运算（如加、减、乘、除）和逻辑运算（如与、或、非）。

2.简单回答下列问题。

(1)为什么计算机内部采用二进制表示信息？

解析：二进制只有 0 和 1 两种状态，这与计算机中的逻辑电路的开关状态一致，使得信息表示和计算非常直接和简单。

二进制数的运算规则简单，使得计算机在进行运算时更加高效。

二进制数的抗干扰能力强，因为它只涉及两种状态，对噪声和干扰的容忍度较高。

(2)既然计算机内部所有信息都用二进制表示，为什么还要用到十六进制或八进制数？

解析：虽然计算机内部所有信息都用二进制表示，但使用十六进制或八进制数是为了方便人类理解和操作。十六进制和八进制数的每一位可以对应多个二进制位，这使得在表示较长的二进制数时，十六进制或八进制数更加简洁，便于阅读和书写。在程序设计和调试中，使用十六进制或八进制数可以更方便地查看和操作内存地址和数据。

(3) 在浮点数的基数和总位数一定的情况下，浮点数的表示范围和精度分别由什么决定？

解析：在浮点数的基数和总位数一定的情况下，浮点数的表示范围和精度分别由阶码和尾数决定。阶码决定了浮点数的范围，即能表示的最大和最小数值；而尾数决定了浮点数的精度，即数值的精确度或小数部分的位数。

(4)为什么要对浮点数进行规格化？有哪两种规格化操作？

解析：对浮点数进行规格化是为了提高浮点数的精度和运算效率。规格化操作包括左规和右规两种。当尾数的绝对值小于规定的最小值时，需要进行右规操作，即将尾数的小数点向右移动，同时增加阶码的值；当尾数的绝对值大于规定的最大值时，需要进行左规操作，

即将尾数的小数点向左移动，同时减少阶码的值。通过规格化操作，可以确保浮点数的尾数满足一定的条件（如最高位为 1），从而提高浮点数的表示范围和精度。

(5)为什么计算机处理汉字时会涉及不同的编码(如输入码、内码、字模码)?说明这些编码中哪些用二进制编码，哪些不用二进制编码，为什么?

解析：计算机处理汉字时会涉及不同的编码，如输入码、内码和字模码，是因为汉字数量庞大，需要有效的编码方式来存储、传输和显示。这些编码中，输入码如拼音码、五笔码等不是用二进制编码的，因为它们是为了方便用户输入汉字而设计的，具有特定的输入规则和逻辑。而内码和字模码则是用二进制编码的。内码是计算机内部用于存储和传输汉字的唯一标识码，需要二进制编码来确保唯一性和计算机识别的准确性。字模码是用于显示汉字的字形信息，通常以二进制图像或点阵形式表示，因此也需要二进制编码。

3.实现下列各数的转换

(1) (25.8125)₁₀= (?)₂= (?)₈= (?)₁₆

解析：(25.8125)₁₀= (11001.1101)₂= (31.64)₈= (19)₁₆

十进制到2进制转换：

将目标基数乘以结果的小数位(最高可达6位数):
floor(25.8125×2⁴) = 413

除以基数，从余下的数字中得到数字。

除法	商数	余数	数位
413/2	206	1	0
206/2	103	0	1
103/2	51	1	2
51/2	25	1	3
25/2	12	1	4
12/2	6	0	5
6/2	3	0	6
3/2	1	1	7
1/2	0	1	8

= (11001.1101)₂

十进制到8进制转换：

将目标基数乘以结果的小数位(最高可达6位数):
floor(25.8125×8²) = 1652

除以基数，从余下的数字中得到数字。

除法	商数	余数	数位
1652/8	206	4	0
206/8	25	6	1
25/8	3	1	2
3/8	0	3	3

= (31.64)₈

(2) (101101.011)₂= (?)₁₀= (?)₈= (?)₁₆

解析：(101101.011)₂= (45.375)₁₀= (55.3)₈= (2D.6)₁₆

2进制到十进制转换： $(101101.011)_2 = (1 \times 2^5) + (0 \times 2^4) + (1 \times 2^3) + (1 \times 2^2) + (0 \times 2^1) + (1 \times 2^0) + (0 \times 2^{-1}) + (1 \times 2^{-2}) + (1 \times 2^{-3}) = (45.375)_{10}$

(3) (4E.C)₁₆= (78.75)₁₀= (1001110.11)₂

16进制到十进制转换:

$$(4E.C)_{16} = (4 \times 16^1) + (14 \times 16^0) + (12 \times 16^{-1}) = (78.75)_{10}$$

4.假定机器数为 8 位(1 位符号, 7 位数值), 写出下列各二进制小数的原码表示。

+0.1001, -0.1001, +1.0, -1.0, +0.010100, -0.010100, +0, -0

解析:

给定机器数为 8 位, 其中 1 位用于符号, 7 位用于数值。原码表示法非常直观, 正数的符号位为 0, 负数的符号位为 1, 其余位直接表示数值。以下是各二进制小数的原码表示:

+0.1001

- 符号位: 0 (正数)
- 数值位: 1001
- 原码表示: 01001000

-0.1001

- 符号位: 1 (负数)
- 数值位: 1001
- 原码表示: 11001000

+1.0

- 符号位: 0 (正数)
- 数值位: 因为小数点后的位数不足 7 位, 所以需要在前面补 0 直到达到 7 位。因此, 数值位为 0000000 (表示 1.0)
- 原码表示: 00000000

-1.0

- 符号位: 1 (负数)
- 数值位: 与+1.0 相同, 为 0000000
- 原码表示: 10000000

+0.010100

- 符号位: 0 (正数)
- 数值位: 010100
- 原码表示: 00101000

-0.010100

- 符号位: 1 (负数)

- 数值位：010100

- 原码表示：10101000

+0

- 符号位：0（正数）

- 数值位：全 0，表示 0

- 原码表示：00000000

-0

- 符号位：1（负数）

- 数值位：全 0，但在原码表示法中，-0 和+0 是相同的，都表示为 00000000

- 原码表示：00000000（注意，在实际计算机系统中，-0 通常会被特殊处理或视为非法表示）

请注意，原码表示法有一个问题，那就是它有两个 0 的表示（+0 和-0）。在实际的计算机系统中，通常会使用补码或其他方法来解决这个问题。

5.假定机器数为 8 位(1 位符号，7 位数值)，写出下列各二进制整数的补码和移码表示。

+1001,-1001,+1,-1,+ 10100,-10100, +0,-0

对于 8 位机器数（1 位符号，7 位数值），我们可以按照补码和移码的规则来求出各二进制整数的表示。

首先，补码的定义是：正数的补码与其原码相同；负数的补码是其绝对值的二进制表示（即原码），然后各位取反，最后加 1。

移码的定义是：在补码的基础上，符号位取反（即正数符号位为 1，负数符号位为 0）。

现在，我们根据这些规则来求出各二进制整数的补码和移码表示：

+1001

- 补码：与原码相同，为 01001

- 移码：符号位取反，为 11001

-1001

- 补码：先求原码（11001），然后取反得 00110，再加 1 得 00111

- 移码：符号位取反，为 00111

+1

- 补码：00001

- 移码：10001

-1

- 补码：原码为 10001，取反得 01110，再加 1 得 01111
- 移码：01111

+10100

- 补码：010100
- 移码：110100

-10100

- 补码：原码为 110100，取反得 001011，再加 1 得 001100
- 移码：001100

+0

- 补码：0000000
- 移码：1000000

-0

- 在补码和移码表示中，通常没有单独的-0表示。在补码系统中，-0和+0是等价的，都表示为全0（0000000）。在移码系统中，由于符号位取反，-0的移码表示会是1000000，但这通常不被视为一个有效的数值表示，因为移码主要用于浮点数的表示，其中-0和+0在大多数系统中被视为相同。

请注意，对于补码和移码，符号位都包含在总的位数中，所以8位表示包括了1位符号位和7位数值位。对于+0和-0，补码系统中它们都是0000000，但在移码系统中，-0没有独立的表示，通常被视为无效或等同于+0的移码表示。

6. 已知 $[x]_{\text{补}}$ ，求 x

(1) $[x]_{\text{补}} = 11100111$

(2) $[x]_{\text{补}} = 1000\ 000$

(3) $[x]_{\text{补}} = 0101\ 0010$

(4) $[x]_{\text{补}} = 1101\ 0011$

解析：

已知补码求原码，我们需要首先确定该补码表示的是正数还是负数，然后根据补码的定义还原出原码。补码的最高位（符号位）为0时，表示正数，此时补码就是原码；补码的最高位为1时，表示负数，此时我们需要对补码求反再加1得到原码。下面针对每一个补码，我们来进行计算：

(1) $[x]_{\text{补}} = 11100111$

- 符号位为 1，表示这是一个负数。我们需要对补码求反再加 1 得到原码。
- 求反：00011000
- 加 1：00011001
- 因此， $x = -00011001$ ，即 -23（十进制）。

(2) $[x]_{\text{补}} = 10000000$

- 符号位为 1，表示这是一个负数。但这里有个特殊情况，补码表示的是-128。在 8 位补码系统中，-128 没有对应的正数形式，因此这是一个特殊的值。
- 因此， $x = -128$ （十进制）。

(3) $[x]_{\text{补}} = 01010010$

- 符号位为 0，表示这是一个正数。补码就是原码。
- 因此， $x = 01010010$ ，即 82（十进制）。

(4) $[x]_{\text{补}} = 11010011$

- 符号位为 1，表示这是一个负数。我们需要对补码求反再加 1 得到原码。
- 求反：00101100
- 加 1：00101101
- 因此， $x = -00101101$ ，即 -43（十进制）。

7. 假定 32 位字长的机器中带符号整数用补码表示，浮点数用 IEEE 754 标准表示，寄存器 R1 和 R2 的内容分别为 R1: 00000H, R2: 8080 00 不同指令对寄存器进行不同的操作，因而，不同指令执行时寄存器内容对应的真值不同。假定执行下列运算指令时,操作数为寄存器 R1 和 R2 的内容，则 R1 和 R2 中操作数的真值分别为多少？

(1)无符号数加法指令

(2)带符号整数乘法指令

(3)单精度浮点数减法指令

解析：

首先，我们需要明确每个指令如何处理寄存器中的内容。寄存器 R1 和 R2 中的内容分别是 00000H 和 80800000H，这些都是十六进制表示。

(1) 无符号数加法指令

- 对于无符号数加法，我们直接将两个数相加。
- R1: 00000000H

- R2: 80800000H
- 相加得:
- 结果 = 80800000H
- 因此, R1 和 R2 中操作数的真值(十进制表示)分别是:
- R1 真值 = 0
- R2 真值 = 2147483648 (因为 80800000H 转换为十进制是 2147483648)

(2) 带符号整数乘法指令

- 对于带符号整数乘法, 首先需要将十六进制数转换为补码表示的带符号整数。
- R1: 00000000H -> 真值 = 0
- R2: 80800000H -> 真值 = -2147483648 (因为是补码表示的负数)
- 然后, 执行乘法运算:
- $0 * (-2147483648) = 0$
- 所以, 尽管乘法运算执行了, 但结果的真值仍然是 0。

(3) 单精度浮点数减法指令

- 对于单精度浮点数减法, 我们需要按照 IEEE 754 标准来解释寄存器中的内容。
- R1: 00000000H -> 0.0 (正零)
- R2: 80800000H -> -Inf (负无穷大, 因为指数位全为 1 且尾数位不全为 0 时表示 NaN 或特殊值, 这里的尾数位为 0, 所以是负无穷大)
- 在 IEEE 754 标准中, 0 减去负无穷大在数学上是没有定义的, 但在大多数实现中, 这样的操作通常会导致结果为正无穷大。但需要注意的是, 这取决于具体的硬件和浮点实现。
- 如果我们按照通常的实现, 结果应该是正无穷大。但由于这取决于具体的实现, 所以严格来说我们不能确定 R1 和 R2 中操作数的真值在执行单精度浮点数减法指令后的确切结果, 只能说结果可能是正无穷大。

7. 假定一台 32 位字长的机器中带符号整数用补码表示, 浮点数用 IEEE 754 标准表示, 寄存器 R1 和 R2 的内容分别为 R1: 00 00 10 8BH, R2: 80 80 10 8BH。不同指令对寄存器进行不同的操作, 因而, 不同指令执行时寄存器内容对应的真值不同。假定执行下列运算指令时, 操作数为寄存器 R1 和 R2 的内容, 则 R1 和 R2 中操作数的真值分别为多少?

- (1) 无符号数加法指令 R1:+108BH R2:+8080108BH
- (2) 带符号整数乘法指令 R1:+108BH R2:-7F7FEF75H

(3) 单精度浮点数减法指令 R1:0000000000000000f 0001000010001011=+0.000000f
0001000010001011B2⁻¹²⁶

R2:100000000100000000001000010001011=-0.0000000001000010001011B2⁻¹²⁶

9. 以下是一个 C 语言程序，用来计算一个数组 a 中每个元素的和。当参数 len 为 0 时，返回值应该是 0，但是在机器上执行时，却发生了存储器访问异常。请问这是什么原因造成的，并说明程序应该如何修改。

```
float sum_elements(float a[], unsigned len)
{
    int i;

    float result = 0;

    for (i = 0; i <= len - 1; i++)

        result += a[i];

    return result;
}
```

解：参数 len 的类型是 unsigned，当 len=0 时，len-1 的结果为 11...1，因此循环体被不断执行，引起数组越界。

应将 len 修改为 int 型或者 for 循环条件改为 i<len

10. 下列几种情况所能表示的数的范围是什么？

(1) 16 位无符号整数

(2) 16 位原码定点小数

(3) 16 位补码定点整数

(4) 下述格式的浮点数(基数为 2，移码的偏置常数为 128，没有隐藏位)

数符（1 位）阶码（8 位移码）尾数（7 位原码数值部分）

解答：对于每种情况，我们可以分别计算出其表示的数的范围。

(1) 16 位无符号整数：在 16 位的情况下，无符号整数的范围为 0 到 $2^{16} - 1$ ，即 0 到 65535。

(2) 16 位原码定点小数：16 位原码定点小数通常采用一个比特表示符号位，剩余比特表示整数和小数部分。假设有 1 位符号位，15 位用于整数和小数部分。那么范围可以计算为从 -1（全部为 1 的补码）到 $1 - 2^{-(15)}$ （最大正数）。所以表示的范围是从 -1 到 $1 - 2^{-(15)}$ 。

(3) 16 位补码定点整数：16 位补码定点整数同样采用一个比特表示符号位，剩余比特表示整数部分。假设有 1 位符号位，15 位用于整数部分。那么范围可以计算为从 -2^{15} （全

部为 1 的补码) 到 $2^{15} - 1$ (全部为 0 的补码)。所以表示的范围是从 -2^{15} 到 $2^{15} - 1$ 。

(4) 下述格式的浮点数: 这个浮点数格式有 1 位符号位, 8 位阶码 (移码), 7 位尾数 (原码数值部分)。阶码的偏置常数为 128, 没有隐藏位。对于阶码, 我们将其看作无符号数。假设阶码的偏置常数为 128, 则阶码的范围是从 0 到 255。尾数的范围是从 0 到 127。因此, 这个浮点数的表示范围是从 $2^{-(128)}$ 到 $(1 - 2^{-(7)}) * 2^{(127)}$ 。

11. 以 IEEE 754 单精度浮点数格式表示下列十进制数。+1.625, -9/16, +1.75, +19, -1/8, 258

(1) ****+1.625****:

- 首先将 +1.625 转换为二进制。整数部分是 1, 小数部分是 0.625。
- 整数部分的二进制是 '1'。
- 小数部分的二进制是 '0.101' (因为 $0.625 = 1/2 + 1/8$)。
- 因此, +1.625 的二进制表示是 '1.101'。
- 将其规范化为科学计数法形式 ' $1.101 * 2^0$ '。
- 在 IEEE 754 格式中, 符号位为 '0', 阶码为 '127' (因为指数偏移是 127, 所以阶码为 $0 + 127 = 127$), 尾数部分为 '101 0000 0000 0000 0000'。
- 因此, +1.625 的 IEEE 754 单精度浮点数表示为 '0 01111111 101000000000000000000000'。

(2) **** -9/16 ****:

- 将 -9/16 转换为二进制。 -9 转换为二进制是 '1001', 16 转换为二进制是 '10000'。
- -9/16 的二进制是 '1001.0000'。
- 将其规范化为科学计数法形式 ' $1.001 * 2^{-3}$ '。
- 在 IEEE 754 格式中, 符号位为 '1', 阶码为 '130' ($3 + 127 = 130$), 尾数部分为 '001000000000000000000000'。

- 因此, -9/16 的 IEEE 754 单精度浮点数表示为 '1 10000010 001000000000000000000000'。

(3) ****+1.75****:

- 将 +1.75 转换为二进制。整数部分是 1, 小数部分是 0.75。
- 整数部分的二进制是 '1'。
- 小数部分的二进制是 '0.11' (因为 $0.75 = 1/2 + 1/4$)。

- 因此, $+1.75$ 的二进制表示是 `'1.11'`。
- 将其规范化为科学计数法形式 `'1.11 * 2^0'`。
- 在 IEEE 754 格式中, 符号位为 `'0'`, 阶码为 `'127'` ($0 + 127 = 127$), 尾数部分为 `'110000000000000000000000'`。

- 因此, $+1.75$ 的 IEEE 754 单精度浮点数表示为 `'0 01111111 110000000000000000000000'`。

(4) ****+19****:

- 将 $+19$ 转换为二进制。 19 的二进制是 `'10011'`。
- 将其规范化为科学计数法形式 `'1.0011 * 2^4'`。
- 在 IEEE 754 格式中, 符号位为 `'0'`, 阶码为 `'131'` ($4 + 127 = 131$), 尾数部分为 `'001100000000000000000000'`。

- 因此, $+19$ 的 IEEE 754 单精度浮点数表示为 `'0 10000111 001100000000000000000000'`。

(5) **** -1/8 ****:

- 将 $-1/8$ 转换为二进制。 $-1/8$ 的二进制是 `'1111.0'`。
- 将其规范化为科学计数法形式 `'1.111 * 2^0'`。
- 在 IEEE 754 格式中, 符号位为 `'1'`, 阶码为 `'127'` ($0 + 127 = 127$), 尾数部分为 `'111000000000000000000000'`。

- 因此, $-1/8$ 的 IEEE 754 单精度浮点数表示为 `'1 01111111 111000000000000000000000'`。

(6) ****258****:

- 将 258 转换为二进制。 258 的二进制是 `'100000010'`。
- 将其规范化为科学计数法形式 `'1.00000010 * 2^8'`。
- 在 IEEE 754 格式中, 符号位为 `'0'`, 阶码为 `'135'` ($8 + 127 = 135$), 尾数部分为 `'000000100000000000000000'`。

- 因此, 258 的 IEEE 754 单精度浮点数表示为 `'0 10000111 000000100000000000000000'`。

12. 设一个变量的值为 4098, 要求分别用 32 位补码整数和 IEEE 754 单精度浮点格式表示该变量(结果用十六进制表示), 并说明哪段二进制序列在两种表示中完全相同, 为什么会相同?

解答: 首先, 让我们分别用 32 位补码整数和 IEEE 754 单精度浮点格式来表示变量值 4098。

分为 000000000000000000000000。

因此，IEEE 754 单精度浮点格式表示为 `1 10011110 000000000000000000000000`。

现在，让我们来分析哪种表示更精确：

- 32 位补码整数表示的值为精确值，因为在整数表示中，每个位都被精确地用于表示数值。

- IEEE 754 单精度浮点格式表示的值为近似值，因为在浮点表示中，精度是受限的，尤其是对于较大的数值，可能会丢失精度或者产生舍入误差。

因此，32 位补码整数表示的值是完全精确的，而 IEEE 754 单精度浮点格式表示的值是近似值。

16.假设某字长为 8 位的计算机中，带符号整数变量采用补码表示，已知 $x=-60$ ， $y=-75$ ， x 和 y 分别存放在寄存器 A 和 B 中。请回答下列问题(要求最终用十六进制表示二进制序列)。

(1)寄存器 A 和 B 中的内容分别是什么？

(2) 若 x 和 y 相加后的结果存放在寄存器 C 中，则寄存器 C 中的内容是什么？运算结果是否正确？加法器最高位的进位 Cout 是什么？溢出标志 OF、符号标志 SF 和零标志 ZF 各是什么？

(3)若 x 和 y 相减后的结果存放在寄存器 D 中，则寄存器 D 中的内容是什么？运算结果是否正确？此时，加法器最高位的进位 Cout 是什么？溢出标志 OF、符号标志 SF 和零标志 ZF 各是什么？

(4)对于带符号整数的减法运算，能否根据借位标志 CF 对两个带符号整数的大小进行比较？

解析：首先，让我们按照题目中所述的要求来回答问题。

(1) 寄存器 A 和 B 中的内容分别是什么？

由于题目中已经说明了采用 8 位补码表示，我们可以将 -60 和 -75 转换为 8 位补码表示。

-60 的补码为：`11000100`

-75 的补码为：`10110101`

所以寄存器 A 中的内容是 `11000100`，寄存器 B 中的内容是 `10110101`。

(2) 若 x 和 y 相加后的结果存放在寄存器 C 中，则寄存器 C 中的内容是什么？运算结果是否正确？加法器最高位的进位 Cout 是什么？溢出标志 OF、符号标志 SF 和零标志 ZF 各是什么？

要将-60和-75相加，我们使用补码的加法规则。在8位补码加法中，如果最高位的进位与次高位的进位不同，则发生溢出。

所以，将-60和-75相加，得到的结果是-135。它的补码是`10001001`。

寄存器C中的内容是`10001001`。

- 运算结果是否正确？正确，因为补码加法给出了正确的结果。
- 加法器最高位的进位 Cout 是什么？由于最高位进位与次高位进位相同，所以进位 Cout 为 0。
- 溢出标志 OF 是什么？由于溢出，所以溢出标志 OF 为 1。
- 符号标志 SF 是什么？由于结果为负数，所以符号标志 SF 为 1。
- 零标志 ZF 是什么？由于结果不为零，所以零标志 ZF 为 0。

(3) 若 x 和 y 相减后的结果存放在寄存器 D 中，则寄存器 D 中的内容是什么？运算结果是否正确？此时，加法器最高位的进位 Cout 是什么？溢出标志 OF、符号标志 SF 和零标志 ZF 各是什么？

要将-60和-75相减，我们使用补码的减法规则。在8位补码减法中，如果最高位的进位与次高位的进位相同，则发生溢出。

所以，将-60和-75相减，得到的结果是15。它的补码是`00001111`。

寄存器D中的内容是`00001111`。

- 运算结果是否正确？正确，因为补码减法给出了正确的结果。
- 加法器最高位的进位 Cout 是什么？由于最高位进位与次高位进位相同，所以进位 Cout 为 0。
- 溢出标志 OF 是什么？由于没有溢出，所以溢出标志 OF 为 0。
- 符号标志 SF 是什么？由于结果为正数，所以符号标志 SF 为 0。
- 零标志 ZF 是什么？由于结果不为零，所以零标志 ZF 为 0。

(4) 对于带符号整数的减法运算，能否根据借位标志 CF 对两个带符号整数的大小进行比较？

在补码表示中，如果两个数相减，且结果为正数，则第一个数大于第二个数；如果结果为负数，则第一个数小于第二个数；如果结果为零，则两个数相等。因此，可以根据减法的结果来比较两个带符号整数的大小。CF 标志位会反映是否发生了借位，但它本身并不能直接用于比较大小。

第三章 程序的转换及机器级表示

一、名词解释

1. 机器语言程序：一种由二进制代码组成的程序，直接由计算机硬件执行的指令集合。
2. 汇编指令：一种由助记符表示的机器指令，与机器语言一一对应，可被汇编器转换成对应的机器语言指令。
3. 汇编语言程序：使用汇编语言编写的程序，以助记符形式表示的指令序列，需要经过汇编器转换为机器语言程序。
4. 汇编助记符：在汇编语言中使用的符号，代表对应的机器指令。
5. 汇编程序：将汇编语言程序转换成机器语言程序的程序，也称为汇编器。
6. 反汇编程序：将机器语言程序转换回汇编语言程序的程序。
7. 机器级代码：直接在计算机硬件上执行的指令，通常是由汇编器将汇编语言转换而来的。
8. CISC：复杂指令集计算机，指一种计算机架构，其指令集包含了多种复杂的指令。
9. RISC：精简指令集计算机，指一种计算机架构，其指令集相对简单，每条指令的执行时间相对较短。
10. 通用寄存器：用于存储临时数据和地址的寄存器，通常用于存储计算过程中的中间结果。
11. 变址寄存器：用于存储地址计算中的偏移量的寄存器，通常用于实现数组、结构体等数据结构的访问。
12. 基址寄存器：用于存储内存地址基址的寄存器，通常与变址寄存器一起使用，用于计算实际内存地址。
13. 栈指针寄存器：用于存储栈顶地址的寄存器，用于实现函数调用和局部变量存储等操作。
14. 指令指针寄存器：用于存储当前正在执行指令的地址的寄存器，通常指向下一条要执行的指令。
15. 标志寄存器：用于存储程序运行状态标志的寄存器，例如零标志、进位标志等。
16. 条件标志(条件码)：标志寄存器中的一部分，用于表示上一条指令的执行结果，通常用于控制条件跳转指令的执行。
17. 寻址方式：指定操作数的寻址方式，例如立即寻址、寄存器寻址、相对寻址等。
18. 立即寻址：直接将操作数的值嵌入到指令中。
19. 寄存器寻址：使用寄存器存储操作数的地址。
20. 相对寻址：使用相对于某个基址的偏移量来寻址。
21. 存储器操作数：指令中的操作数直接引用存储器中的数据。
22. 实地址模式：CPU 直接使用物理地址访问内存的模式。
23. 保护模式：一种 CPU 工作模式，操作系统可在其中对内存和其他资源进行保护和管理。
24. 有效地址：执行指令时计算出的操作数在内存中的真实地址。
25. 比例变址：根据一个基址寄存器和一个比例系数来计算操作数的地址。

- 26. 非比例变址：仅根据一个基址寄存器来计算操作数的地址。
- 27. 比例系数(比例因子)：用于比例变址计算的一个数值。
- 28. MMX 指令：Intel 处理器中用于多媒体处理的指令集。
- 29. SSE 指令集：Intel 处理器中的一种扩展指令集，用于实现单指令多数据（SIMD）操作。
- 30. SIMD：单指令多数据，一种并行计算的技术，能够同时处理多个数据。
- 31. 多媒体扩展通用寄存器：用于 SIMD 指令集操作的寄存器。
- 32. 栈(Stack)：一种后进先出（LIFO）的数据结构，用于临时存储函数调用的上下文和局部变量。
- 33. 调用者保存寄存器：在函数调用过程中，由调用者负责保存和恢复的寄存器。
- 34. 被调用者保存寄存器：在函数调用过程中，由被调用者负责保存和恢复的寄存器。
- 35. 帧指针寄存器：用于指向当前函数调用的栈帧的寄存器。
- 36. 当前栈帧：当前函数调用的栈帧，存储了函数的参数、局部变量和返回地址等信息。
- 37. 按值传递参数：将参数的值传递给函数。
- 38. 按地址传递参数：将参数的地址传递给函数，函数可以直接访问参数的值。
- 39. 嵌套调用：在一个函数内部调用另一个函数的过程。
- 40. 递归调用：函数直接或间接地调用自身的过程。

二、简单题

(1)一条机器指令通常由哪些字段组成?各字段的含义分别是什么?

解答：一条机器指令通常由以下字段组成：

- 操作码 (Opcode)：指示该指令的操作类型，如加法、乘法等。
- 操作数 (Operand)：指令要操作的数据或数据的存储地址。
- 寻址方式 (Addressing Mode)：指定如何获取操作数的地址。
- 操作数长度 (Operand Length)：指定操作数的长度，通常以位或字节为单位。

(2)将一个高级语言源程序转换成计算机能直接执行的机器代码通常需要哪几个步骤?

解析：将高级语言源程序转换成计算机能直接执行的机器代码通常需要以下步骤：

- 词法分析 (Lexical Analysis)：将源代码分割成词法单元。
- 语法分析 (Syntax Analysis)：将词法单元组合成语法结构。
- 语义分析 (Semantic Analysis)：检查语法结构是否符合语言规范。
- 代码生成 (Code Generation)：将语法结构翻译成目标机器的机器代码。
- 代码优化 (Code Optimization)：对生成的机器代码进行优化，以提高执行效率。

(3)IA-32 中的逻辑运算指令如何生成条件标志?移位指令可能会改变哪些条件标志?

解析：在 IA-32 中，逻辑运算指令（如 AND、OR、XOR）会根据操作数的结果生成条件标志（如零标志 ZF、进位标志 CF 等）。移位指令（如 SHL、SHR）可能会改变零标志 ZF、进位标志 CF 和溢出标志 OF。

(4)执行条件跳转指令时所用到的条件标志信息从何而来?请举例说明。

解析：执行条件跳转指令时所用到的条件标志信息来自于上一条影响条件标志的指令。例如，执行 JZ（如果零标志为 1 则跳转）指令时，需要根据上一条指令的执行结果来确定是否跳转。

(5) 无条件跳转指令和调用指令的相同点和不同点是什么？

解析：无条件跳转指令和调用指令的相同点是它们都可以改变程序执行的流程，使程序跳转到指定的地址执行。不同点在于无条件跳转指令只是简单地改变程序计数器的值，而调用指令还会将当前的程序状态（如返回地址）保存到栈中，以便后续返回。

(6) 按值传递参数和按地址传递参数两种方式有哪些不同点？

解析：按值传递参数和按地址传递参数的不同点在于：

- 按值传递参数会将参数的值复制给函数的形式参数，因此函数内部对形式参数的修改不会影响实际参数的值；
- 按地址传递参数则会将参数的地址传递给函数，函数内部对形式参数的修改会影响实际参数的值。

(7) 为什么在递归深度较深时递归调用的时间开销和空间开销都会较大？

解析：当递归深度较深时，递归调用的时间开销和空间开销都会较大是因为每次递归调用都会占用额外的栈空间，并且频繁的函数调用和返回会增加时间开销。

(8) 为什么数据在存储器中最好按地址对齐方式存放？

解析：数据在存储器中最好按地址对齐方式存放是因为许多计算机体系结构对数据的访问有地址对齐的要求。如果数据没有按照正确的地址对齐方式存放，可能会导致额外的内存访问，降低访问效率。

3. 对于以下 AT&T 格式汇编指令，根据操作数的长度确定对应指令助记符中的长度后缀，并说明每个操作数的寻址方式。

- (1) `mov 8(%ebp, %ebx, 4), %ax`
- (2) `mov %al, 12(%ebp)`
- (3) `add (, %ebx, 4), %ebx`
- (4) `or (%ebx), %dh`
- (5) `push $0xF8`
- (6) `mov $0xFFFF0, %eax`
- (7) `test %cx, %cx`
- (8) `lea 8(%ebx, %esi), %eax`

解析：

对于给出的 AT&T 格式汇编指令，我们将根据操作数的长度确定对应指令助记符中的长度后缀，并说明每个操作数的寻址方式。

- (1) ``mov 8(%ebp, %ebx, 4), %ax``
 - 指令助记符：``mov``
 - 操作数长度后缀：``w``（表示 word，16 位）
 - 操作数 1：``8(%ebp, %ebx, 4)``，寻址方式为 ``[base + index * scale + displacement]``。这里 ``8(%ebp, %ebx, 4)`` 表示从内存地址 ``[ebp + ebx * 4 + 8]`` 中读取数据。
 - 操作数 2：``%ax``，寻址方式为寄存器直接。
- (2) ``mov %al, 12(%ebp)``
 - 指令助记符：``mov``

- 操作数长度后缀: ``b`` (表示 byte, 8 位)
- 操作数 1: ``%a1``, 寻址方式为寄存器直接。
- 操作数 2: ``12(%ebp)``, 寻址方式为 ``[base + displacement]``。这里 ``12(%ebp)`` 表示从内存地址 ``[ebp + 12]`` 中读取数据。

(3) ``add(, %ebx, 4), %ebx``

- 指令助记符: ``add``
- 操作数长度后缀: 根据默认寄存器长度确定, 这里 ``%ebx`` 是 32 位寄存器, 因此默认为 ``l`` (long, 32 位)
- 操作数 1: ``(, %ebx, 4)``, 寻址方式为 ``[index * scale]``。这里 ``(, %ebx, 4)`` 表示从内存地址 ``[ebx * 4]`` 中读取数据。

- 操作数 2: ``%ebx``, 寻址方式为寄存器直接。

(4) ``or(%ebx), %dh``

- 指令助记符: ``or``
- 操作数长度后缀: ``b`` (表示 byte, 8 位)
- 操作数 1: ``(%ebx)``, 寻址方式为 ``[ebx]``, 从内存地址 ``[ebx]`` 中读取数据。
- 操作数 2: ``%dh``, 寻址方式为寄存器直接。

(5) ``push $0xF8``

- 指令助记符: ``push``
- 操作数长度后缀: N/A, 立即数直接推送到栈中, 大小与操作数大小无关。
- 操作数: ``$0xF8``, 即立即数 ``0xF8``。

(6) ``mov $0xFFFF0, %eax``

- 指令助记符: ``mov``
- 操作数长度后缀: 默认为 ``l`` (long, 32 位)
- 操作数 1: ``$0xFFFF0``, 即立即数 ``0xFFFF0``。
- 操作数 2: ``%eax``, 寻址方式为寄存器直接。

(7) ``test %cx, %cx``

- 指令助记符: ``test``
- 操作数长度后缀: 根据 ``%cx`` 的寄存器大小确定, 默认为 ``w`` (word, 16 位)
- 操作数 1: ``%cx``, 寄存器直接。
- 操作数 2: ``%cx``, 寄存器直接。

(8) ``lea 8(%ebx, %esi), %eax``

- 指令助记符: ``lea`` (Load Effective Address)
- 操作数长度后缀: 默认为 ``l`` (long, 32 位)
- 操作数 1: ``8(%ebx, %esi)``, 寻址方式为 ``[ebx + esi + 8]``, 将有效地址加载到 ``%eax`` 寄存器中。
- 操作数 2: ``%eax``, 寄存器直接。

4. 使用汇编器处理以下各行 AT&T 格式代码时都会产生错误, 请说明每一行存在什么错误。

(1) `movl 0xFF, ( %eax)`

```

(2) movb %ax, 12(%ebp)
(3) addl %ecx, $0xF0
(4) orw $0xFFFF0, (%ebx)
(5) addb $0xF8, (%dl)
(6) movl %bx, %eax
(7) andl %esi, %esx
(8) movw 8(%ebp, , 4), %ax

```

解析：

当使用汇编器处理以下 AT&T 格式代码时会产生错误，以下是每一行存在的错误：

(1) ``movl 0xFF, (%eax)``

错误：在内存寻址时，地址应该是有效的内存地址，但这里 ``(%eax)`` 之间多了一个空格，应该是 ``(%eax)`` 才是正确的寻址方式。另外，立即数 ``0xFF`` 的长度是一个字节，但这里使用的是 ``movl`` 指令，该指令表示操作数是一个 32 位长的值，因此需要使用 ``movb`` 指令。

(2) ``movb %ax, 12(%ebp)``

错误：在将 ``%ax`` 寄存器内容移动到内存时，``%ax`` 是一个 16 位寄存器，但是目的操作数 ``12(%ebp)`` 表示的是一个字节。这意味着 16 位的 ``%ax`` 中的值不能完全放入一个字节的目的地位置中，应该使用 ``movw`` 指令来将 16 位的数据移动到内存。

(3) ``addl %ecx, $0xF0``

错误：在 ``addl`` 指令中，第一个操作数应该是目的操作数，但是这里 ``$0xF0`` 是一个立即数，应该放在第二个操作数的位置上。正确的语法应该是 ``addl $0xF0, %ecx``。

(4) ``orw $0xFFFF0, (%ebx)``

错误：立即数 ``$0xFFFF0`` 超出了 16 位寄存器 ``%ebx`` 可以表示的范围（16 位）。应该使用适当大小的立即数来避免溢出。

(5) ``addb $0xF8, (%dl)``

错误：``%dl`` 是一个 8 位的寄存器，而 ``addb`` 指令是用来对 8 位操作数进行加法运算的。因此，这里使用 ``%dl`` 作为目的操作数是不正确的。应该选择一个 32 位或者 16 位的寄存器作为目的操作数。

(6) ``movl %bx, %eax``

错误：``%bx`` 是一个 16 位寄存器，而 ``%eax`` 是一个 32 位寄存器，它们的大小不匹配。应该使用 ``movw`` 指令来移动 ``%bx`` 的值到 ``%eax``，或者使用 ``movzwl`` 来将 ``%bx`` 的值扩展为 32 位。

(7) ``andl %esi, %esx``

错误：这里 ``%esx`` 应该是 ``%esi``，因为 ``%esi`` 是一个 32 位寄存器，而 ``%esx`` 似乎是一个错误的操作数。

(8) ``movw 8(%ebp, , 4), %ax``

错误：在计算内存地址时，括号中的表达式中缺少一个索引寄存器。应该是 ``movw 8(%ebp, %index, 4), %ax``，其中 ``%index`` 是一个索引寄存器。

5. 假设变量 x 和 ptr 的类型声明如下：

```
src_type x;
dst_type* ptr;
```

这里，src_type 和 dst_type 是用 typedef 声明的数据类型。有以下几个 C 语言赋值语句：

```
*ptr=( dst_type) x;
```

若 x 存储在寄存器 EAX 或 AX 或 AL 中，ptr 存储在寄存器 EDX 中，则对于表 3.12 中给出的 src_type 和 dst_type 的类型组合，写出实现上述赋值语句的机器级代码。要求用 AT&T 格式表示机器级代码。

对于给出的不同 src_type 和 dst_type 的组合，下面是实现赋值语句 `\*ptr = (dst\_type)x;` 的机器级代码，使用 AT&T 格式表示：

表 3.12 题 5 用表

src_type	dst_type	机器级表示
char	int	
int	char	
int	unsigned	
short	int	
unsigned char	unsigned	
char	unsigned	
int	int	

(1) `src\_type(char)`、`dst\_type(int)` 的机器级表示为：

```
movsbl %al, %eax # 将 x 的值从 AL 扩展为 32 位有符号整数，并存储到 EAX
movl %eax, (%edx) # 将 EAX 中的值存储到 ptr 所指向的内存地址中
```

(2) `src\_type(int)`、`dst\_type(char)` 的机器级表示为：

```
movb %al, (%edx) # 将 x 的低字节存储到 ptr 所指向的内存地址中
```

(3) `src\_type(int)`、`dst\_type(unsigned)` 的机器级表示为：

```
movl %eax, (%edx) # 将 x 的值存储到 ptr 所指向的内存地址中
```

(4) `src\_type(short)`、`dst\_type(int)` 的机器级表示为：

```
movswl %ax, %eax # 将 x 的值从 AX 扩展为 32 位有符号整数，并存储到 EAX
movl %eax, (%edx) # 将 EAX 中的值存储到 ptr 所指向的内存地址中
```

(5) `src\_type(unsigned char)`、`dst\_type(unsigned)` 的机器级表示为：

```
movzbl %al, %eax # 将 x 的值从 AL 扩展为 32 位无符号整数，并存储到 EAX
movl %eax, (%edx) # 将 EAX 中的值存储到 ptr 所指向的内存地址中
```

(6) `src\_type(char)`、`dst\_type(unsigned)` 的机器级表示为：

```
movsbl %al, %eax # 将 x 的值从 AL 扩展为 32 位有符号整数，并存储到 EAX
movl %eax, (%edx) # 将 EAX 中的值存储到 ptr 所指向的内存地址中
```

(7) `src\_type(int)`、`dst\_type(int)` 的机器级表示为：

```
movl %eax, (%edx) # 将 x 的值存储到 ptr 所指向的内存地址中
```

6. 假设某个 C 语言函数 func 的原型声明如下：

```
void func( int * xptr, int * yptr, int * zptr);
```

函数 func 的过程体对应的机器级代码用 AT&T 汇编形式表示如下：

```
(1) Movl 8( %ebp), %eax
(2) Movl 12(%ebp), %ebx
(3) movl 16( %ebp), %ecx
(4) Movl (%ebx), %edx
(5) Movl ( %ecx), %esi
(6) Movl ( %eax), %edi
(7) Movl %edi, ( %ebx)
(8) Movl %edx, ( %ecx)
(9) Movl %esi, ( %eax)
```

回答下列问题或完成下列任务。

(1) 在过程体开始时三个入口参数对应实参所存放的存储单元地址是什么(提示:当前栈帧底部由帧指针寄存器 EBP 指示)?

(2) 根据上述机器级代码写出函数 func 的 C 语言代码。

解析:

(1) 在过程体开始时, 三个入口参数对应实参所存放的存储单元地址如下:

- `xptr` 的地址存放在 `8(%ebp)` 中
- `yptr` 的地址存放在 `12(%ebp)` 中
- `zptr` 的地址存放在 `16(%ebp)` 中

这是因为在函数调用过程中, 参数是按顺序被压入栈中, 而函数的帧指针 `%ebp` 指向当前栈帧的底部, 所以在栈上偏移量为 `8(%ebp)`、`12(%ebp)`、`16(%ebp)` 的位置分别存储着相应参数的地址。

(2) 根据上述机器级代码, 函数 `func` 的 C 语言代码可以写成:

```
void func(int *xptr, int *yptr, int *zptr) {
    int x = *yptr;
    int y = *zptr;
    int z = *xptr;

    *xptr = y;
    *yptr = z;
    *zptr = x;
}
```

这段 C 代码实现了与给定机器级代码相对应的功能。

9. 假设函数 operate 的部分 C 代码如下:

```
1 int operate(int x, int y, int z, int k){
2 int v=____;
3 return v;
4 }
```

以下汇编代码用来实现第 2 行语句的功能，请写出每条汇编指令的注释，并根据以下汇编代码，填写 operate 函数缺失的部分。

```
(1) movl 12(%ebp), %ecx
(2) sall $8, %ecx
(3) movl 8(%ebp), %eax
(4) movl 20(%ebp), %edx
(5) imull %edx, %eax
(6) movl 16(%ebp), %edx
(7) andl $65520, %edx
(8) addl %ecx, %edx
(9) subl %edx, %eax
```

解析：

根据给出的汇编代码，我们可以添加注释，并填写 `operate` 函数缺失的部分：

```
1 int operate(int x, int y, int z, int k){
2     int v=____; // v 的初始值
3     return v;
4 }
```

汇编指令的注释如下：

1. `movl 12(%ebp), %ecx`：将参数 `z` 的值加载到 `%ecx` 寄存器中。
2. `sall \$8, %ecx`：将 `%ecx` 寄存器中的值左移 8 位（相当于乘以 256）。
3. `movl 8(%ebp), %eax`：将参数 `x` 的值加载到 `%eax` 寄存器中。
4. `movl 20(%ebp), %edx`：将参数 `k` 的值加载到 `%edx` 寄存器中。
5. `imull %edx, %eax`：将 `%edx` 寄存器中的值与 `%eax` 寄存器中的值相乘，并将结果存储到 `%eax` 中。
6. `movl 16(%ebp), %edx`：将参数 `y` 的值加载到 `%edx` 寄存器中。
7. `andl \$65520, %edx`：将 `%edx` 寄存器中的值与 65520 (0xFF00) 进行按位与运算。
8. `addl %ecx, %edx`：将 `%ecx` 寄存器中的值加到 `%edx` 寄存器中。
9. `subl %edx, %eax`：将 `%edx` 寄存器中的值从 `%eax` 寄存器中减去，并将结果存储到 `%eax` 中。

根据这些注释，我们可以填写缺失的部分，即变量 `v` 的初始化：

```
int v = (z << 8) + x * k - (y & 0xFF00);
```

10. 假设函数 product 的 C 语言代码如下，其中 num_type 是用 typedef 声明的数据类型。

```
(1) void product( num_type* d, unsigned x, num_type y){
(2)     *d = x*y;
(3) }
```

函数 product 的过程体对应的主要汇编代码如下：

- (1) `Movl 12(%ebp), %eax`
- (2) `Movl 20(%ebp), %ecx`
- (3) `imull %eax, %ecx`
- (4) `Mull 16(%ebp)`
- (5) `Leal (%ecx, %edx), %edx`
- (6) `Movl 8(%ebp), %ecx`
- (7) `Movl %eax, (%ecx)`
- (8) `Movl %edx, 4(%ecx)`

请给出上述每条汇编指令的注释，并说明 `num_type` 是什么类型。

解析：

下面是给出的汇编代码的注释以及关于 `num_type` 类型的说明：

1. `Movl 12(%ebp), %eax` // 将参数 `y` 的值加载到 `%eax` 寄存器中
2. `Movl 20(%ebp), %ecx` // 将参数 `x` 的值加载到 `%ecx` 寄存器中
3. `imull %eax, %ecx` // 将 `%ecx` 寄存器中的值与 `%eax` 寄存器中的值相乘，结果存储在 `%ecx` 中
4. `Mull 16(%ebp)` // 将参数 `d` 的地址加载到 `%edx: %eax` 寄存器中，同时将 `%ecx` 寄存器中的值与参数 `d` 所指向的内存地址处的值相乘，结果存储在 `%edx: %eax` 中
5. `Leal (%ecx, %edx), %edx` // 计算 `%ecx` 寄存器中的值加上 `%edx` 寄存器中的值的地址，结果存储在 `%edx` 寄存器中
6. `Movl 8(%ebp), %ecx` // 将参数 `d` 的地址加载到 `%ecx` 寄存器中
7. `Movl %eax, (%ecx)` // 将 `%eax` 寄存器中的值存储到 `%ecx` 寄存器中指向的内存地址中
8. `Movl %edx, 4(%ecx)` // 将 `%edx` 寄存器中的值存储到 `%ecx` 寄存器中指向的内存地址加上 4 字节处

根据汇编代码，`num_type` 是一个指向某种数据类型的指针。

第四章 可执行文件的生成与加载执行

一、名词解释

链接：在计算机编程中，链接是指将多个源文件或目标文件中的代码和数据组合成一个可执行文件或共享库文件的过程。

可重定位文件：包含了程序的代码和数据，但是还没有完成地址的最终分配，可以在链接时进行地址的重定位，得到可执行文件。

可执行文件：经过链接之后，完成了地址的最终分配，可以直接在操作系统上执行的文件。

符号解析：在链接过程中，将符号（如函数名、变量名）与其在程序中的地址或者其他符号相关联的过程。

重定位：将程序中的符号引用与其真正的地址或者其他符号关联起来的过程。

ELF 头：Executable and Linkable Format 头，是一种用于描述可执行文件和共享库文件结构的数据结构。

节头表：包含了可执行文件或共享库文件中各个节（**section**）的信息，如起始地址、大小等。

程序头表：包含了可执行文件或共享库文件中各个程序段（**segment**）的信息，如加载地址、大小等。

只读代码段：存放程序的代码，只允许读取操作，不允许写入操作。

可读/写数据段：存放程序的数据，允许读取和写入操作。

全局符号：在整个程序中都可见的符号，可以被其他文件引用。

外部符号：在当前文件中未定义，但在其他文件中定义的符号。

本地符号：仅在当前文件中可见的符号，对其他文件不可见。

强符号：在链接过程中，优先选择的符号。

COMMON 符号：未被初始化的全局变量或未定义的外部变量的符号。

静态链接：在编译时将所有的库文件链接到可执行文件中的链接方式。

共享库文件：包含了可执行文件所需的代码和数据，可以在多个进程间共享使用的文件。

动态链接：在程序运行时将共享库文件链接到内存中的链接方式。

动态链接器：负责在程序运行时将共享库文件链接到内存中的系统程序。

动态链接库：包含了可执行文件所需的代码和数据，可以在程序运行时被动态链接到内存中使用的库文件。

进程：计算机中正在运行的程序的实例，是操作系统进行资源分配和调度的基本单位。

进程描述符：操作系统中用于描述进程的数据结构。

命令行解释程序：接收用户输入的命令行，并将其解释为操作系统能够执行的命令的程序。

指令周期：CPU 执行一条指令所经历的时间周期。

指令译码器：负责将指令解码成对应的操作控制信号的硬件模块。

内部异常：由程序执行过程中的错误或不正常情况引起的异常。

外部中断：由外部设备（如硬件中断、时钟中断）引起的中断。

数据通路：CPU 中负责数据传输和处理的部件的集合。

执行部件：CPU 中负责执行指令的部件，包括算术逻辑单元(ALU)等。

功能部件：CPU 中用于实现特定功能的部件，如乘法器、除法等。

控制器：CPU 中负责控制整个运算过程的部件。

时钟信号：CPU 内部的时钟信号，用于同步各个部件的工作。

指令流水线：CPU 中用于提高指令执行速度的一种技术，将指令执行过程划分为多个阶段并行处理。

二、简答题

(1)如何将多个 C 程序模块组合生成可执行文件?简述从源程序到可执行文件的转换过程。

将多个 C 程序模块组合生成可执行文件的过程主要包括编译（**Compilation**）、汇编（**Assembly**）、链接（**Linking**）三个阶段。

1. 编译阶段：对每个 C 程序模块进行编译，生成对应的目标文件（**Object File**），目标文件中包含了汇编代码但还没有进行地址分配。编译器将源代码转换成汇编代码，并生成目标文件。

2. 汇编阶段：对每个目标文件进行汇编，生成对应的目标文件的汇编文件（**Assembly File**），汇编器将汇编代码转换成机器语言代码，生成可重定位文件（**Relocatable File**）。

3. 链接阶段：将所有的可重定位文件以及所需的库文件进行链接，生成最终的可执行文件。链接器会将各个模块的目标文件组合起来，解析符号引用并进行地址重定位，最终生成一个完整的可执行文件。

简而言之，从源程序到可执行文件的转换过程包括编译、汇编和链接三个阶段，分别将源代码转换为目标文件、汇编文件和最终的可执行文件。在链接阶段，通过解析符号引用和地址重定位，将各个模块组合成一个完整的可执行文件。

(2)可重定位目标文件和可执行目标文件的主要差别是什么？

可重定位目标文件是经过编译和汇编，但尚未进行链接的文件，其中的地址还没有确定。

可执行目标文件是经过编译、汇编和链接，地址已经最终确定，可以直接在操作系统上执行的文件。

(3)链接器主要完成哪两方面的工作？

符号解析：解析各个模块之间的符号引用，确定符号对应的地址。

重定位：将各个模块的地址关联起来，生成最终的可执行文件。

(4)可重定位文件的.text 节、.rodata 节、.data 节和.bss 节中分别主要包含什么信息？

.text 节：存放程序的代码。

.rodata 节：存放只读数据，如常量字符串等。

.data 节：存放已初始化的全局和静态变量。

.bss 节：存放未初始化的全局和静态变量。

(5)可执行目标中有哪两种可装入段？哪些节组合成只读代码段？哪些节组合成可读/写数据段？

只读代码段：由.text 节和.rodata 节组合而成。

可读/写数据段：由.data 节和.bss 节组合而成。

(6)加载可执行文件时，加载器根据其中哪个表的信息对可装入段进行映射？

加载器根据程序头表（Program Header Table）中的信息对可装入段进行映射。

(7)静态链接和动态链接的差别是什么？

静态链接将所有的库文件链接到可执行文件中，在编译时就已经确定了所有的符号引用；动态链接是在程序运行时将共享库文件链接到内存中，允许多个程序共享同一个库文件，需要动态链接器在运行时解析符号引用。

(8)在可执行文件中将可装入段映射到虚拟地址空间，以形成每个进程独立的虚拟地址空间，这种做法有什么好处？

将可装入段映射到虚拟地址空间可以形成每个进程独立的虚拟地址空间，避免了进程间的地址冲突，提高了系统的安全性和稳定性。

(9)简述通过 shell 命令行解释程序进行程序加载的过程。

通过 shell 命令行解释程序进行程序加载的过程大致包括以下几个步骤：

用户输入命令行。

shell 解释命令行，识别要执行的程序。

shell 创建一个新的进程来执行该程序。

加载器加载可执行文件到内存中。

加载器执行程序入口点。

(10)CPU 的基本组成和基本功能各是什么？

CPU 的基本组成包括算术逻辑单元（ALU）、控制单元（Control Unit）、寄存器组。其基本功能是执行指令、进行算术和逻辑运算、控制程序的执行流程。

(11)如何控制一条指令执行结束后能够接着另一条指令执行？

控制一条指令执行结束后能够接着另一条指令执行是通过程序计数器（PC）的自增实现的，每执行一条指令，PC 自动加一，指向下一条指令的地址。

(12)通常一条指令的执行要经过哪些步骤？每条指令的执行步骤都一样吗？

一条指令的执行通常经过取指令、译码、执行、访存、写回这几个步骤。每条指令的执行步骤并不完全相同，不同的指令可能会有不同的执行路径和需要访问的数据。

(13)流水线方式下，一条指令的执行时间缩短了还是加长了？程序的执行时间缩短了还是加长了？

在流水线方式下，一条指令的执行时间缩短了，因为多条指令可以并行执行；但程序的执行时间可能会加长，因为流水线的阶段之间可能存在数据相关性或者控制相关性，导致流水线停顿或者冒险，从而影响程序的执行效率。

(14)具有什么特征的指令系统易于实现指令流水线？

易于实现指令流水线的特征包括：指令长度固定、指令之间无数据相关性、指令之间无控制相关性、指令之间的执行步骤相对简单等。

一、名词解释

随机存取存储器(RAM): RAM (Random Access Memory) 是一种临时存储设备, 用于存储计算机当前正在处理的数据和程序。它可以快速地读取和写入数据, 但一旦电源关闭, 其中的信息就会丢失。

只读存储器(ROM): ROM (Read-Only Memory) 是一种数据存储设备, 一旦数据被写入, 就不能被修改或删除。它通常用于存储计算机的基本启动指令或固件。

易失性存储器: 易失性存储器是指当电源关闭后, 数据会丢失的存储设备。RAM 就是一种易失性存储器。

存储阵列(Bank): 在内存设计中, 存储阵列 (Bank) 指的是一组物理上相互独立的内存模块, 每个模块都有自己的地址和数据总线。这允许处理器并行访问多个 Bank, 从而提高性能。

编址单位: 编址单位指的是在内存或其他存储设备中用于标识和访问单个数据元素的最小单位。

记忆单元(Cell): 记忆单元是存储设备中最基本的存储单位, 用于存储一个二进制位 (bit) 的信息。

存储周期: 存储周期是指存储器进行连续两次独立的存储器操作 (如读或写) 之间所需的最小时间间隔。

编址方式: 编址方式是指如何给存储设备中的每个单元分配地址的方法。不同的编址方式会影响存储器的访问速度和效率。

静态 RAM(SRAM): SRAM (Static Random Access Memory) 是一种 RAM 类型, 它的存储单元使用双稳态电路来存储数据, 不需要刷新, 因此速度较快但功耗较高。

动态 RAM(DRAM): DRAM (Dynamic Random Access Memory) 是另一种 RAM 类型, 它的存储单元使用电容来存储数据, 由于电容会漏电, 所以需要定期刷新来保持数据。

闪存(Flash 存储器): 闪存 (Flash Memory) 是一种非易失性存储设备, 它使用浮栅晶体管来存储数据。闪存常用于 USB 驱动器、SSD 和某些类型的移动设备中。

SDRAM: SDRAM (Synchronous Dynamic Random Access Memory) 是一种同步动态 RAM, 其内部命令的发送与接收采用时钟同步方式。它广泛应用于现代计算机系统中。

行地址选通信号(RAS): 在 DRAM 中, 行地址选通信号 (Row Address Strobe) 用于指定要访问的存储单元的行地址。

列地址选通信号(CAS): 列地址选通信号 (Column Address Strobe) 在 DRAM 中用于指定要访问的存储单元的列地址。

磁盘驱动器: 磁盘驱动器是用于读取和写入磁盘 (如硬盘) 数据的设备。

寻道时间: 寻道时间是指磁盘驱动器将读写头移动到指定磁道所需的时间。

旋转(等待)时间: 旋转时间 (或等待时间) 是指从磁盘驱动器开始寻找数据到数据所在扇区旋转到读写头下方所需的时间。

数据传输率: 数据传输率是指存储设备在单位时间内可以传输的数据量。

固态硬盘(SSD): 固态硬盘是一种使用闪存技术存储数据的存储设备, 它比传统的机械硬盘更快、更耐用。

时间局部性: 时间局部性是指如果一个信息项被访问, 那么在未来不久它很可能再次被

访问。

cache 行(槽): 在计算机缓存中, cache 行(或槽)是缓存中用于存储数据的基本单元。

命中率: 命中率是指缓存中访问成功的次数与总访问次数的比例。

磁盘控制器: 磁盘控制器是管理和控制磁盘驱动器操作的硬件或软件组件。

平均存取时间: 平均存取时间是指从发出数据访问请求到数据实际可用所需的平均时间。

空间局部性: 空间局部性是指如果程序访问了某个存储单元, 那么在不久的将来, 其附近的存储单元也很可能被访问。

主存块: 在缓存和主存之间, 主存块是主存中可以被作为一个单元整体地传输到缓存中的一组数据。

命中时间: 命中时间是指缓存命中时, 从发出数据请求到数据实际可用所需的时间。

缺失损失: 缺失损失是指当缓存未命中时, 从慢速存储器(如主存)获取数据所需的额外时间和开销。

直接映射 (Direct Mapping): 在计算机存储体系中, 直接映射是一种缓存映射方式。在这种方式下, 每个主存页面直接映射到缓存中的特定页面。这种方法简单但可能导致缓存冲突, 降低缓存效率。

FIFO 算法 (First-In-First-Out Algorithm): FIFO 算法是一种缓存置换算法, 也被称为先进先出算法。当缓存满时, 最早进入缓存的数据项会被首先移出, 以腾出空间给新进入的数据项。

回写法 (Write Back): 在缓存系统中, 回写法是一种数据更新策略。当 CPU 修改缓存中的数据时, 修改并不会立即写回到主存中, 而是等到缓存中的数据块被替换或显式地执行写回操作时, 才将修改后的数据写回主存。

物理地址: 物理地址是内存中各存储单元的编号, 即存储单元的真实地址, 它是可识别、可寻址并实际存在的。它是 CPU 访问内存时使用的地址。

未缓存页: 未缓存页是指在虚拟内存中, 但尚未被加载到物理内存中的页面。这些页面在需要时会被加载到物理内存中。

页表基址寄存器: 页表基址寄存器用于存放当前运行程序的页表的起始地址。当 CPU 需要访问某个虚拟地址时, 会与页表基址寄存器中的值拼接, 从而找到对应的页表项。

快表 (TLB, Translation Lookaside Buffer): 快表是一种缓存机制, 用于存储虚拟地址到物理地址的转换表的一部分。当 CPU 需要解析虚拟地址时, 首先会查找快表, 如果找到对应的转换条目, 则可以直接进行地址转换, 从而加速地址解析过程。

用户态: 用户态是操作系统的一种运行模式, 在这种模式下, 程序运行在非特权级别, 不能执行特权指令, 也不能直接访问系统资源。

全相联映射: 全相联映射是一种缓存映射方式, 在这种方式下, 主存中的任一页面可以映射到缓存中的任一页面位置, 没有固定的对应关系。

LRU 算法 (Least Recently Used Algorithm): LRU 算法是一种常用的缓存置换算法。该算法认为最近最少使用的数据项在未来被访问的可能性最小, 因此当缓存满时, 会选择最久未使用的数据项进行替换。

组相联映射: 组相联映射是介于直接映射和全相联映射之间的一种缓存映射方式。它将主存页面分成若干组, 每组内的页面映射到缓存中的固定位置, 但组内各页面可以映射到缓

存中的不同位置。

LRU 位：LRU 位通常用于跟踪缓存中数据项的使用情况，以便实现 LRU 算法。在某些实现中，每次数据项被访问时，其 LRU 位都会被更新，以便在需要替换时能够找到最久未使用的数据项。

替换策略：替换策略决定了当缓存满时，如何选择数据项进行替换。不同的替换策略会影响缓存的效率和性能。

cache 写策略：Cache 写策略决定了 CPU 对 Cache 中的数据进行修改时，如何更新主存中的数据。常见的写策略包括写回法和写通法。

通写法 (Write Through)：通写法是另一种缓存写策略。与回写法不同，当 CPU 修改缓存中的数据时，修改会立即写回到主存中，确保主存中的数据始终是最新的。

虚拟地址 (逻辑地址)：虚拟地址是用户程序中的地址表示方式，它并不直接对应物理内存中的实际地址。在程序执行过程中，虚拟地址需要通过地址映射或地址重定位转换为物理地址。

虚拟页号：虚拟页号是虚拟地址中的一部分，用于标识虚拟内存中的页面。

页框 (页帧)：页框是物理内存中的一个固定大小的块，用于存储从虚拟内存中加载的数据。

请求分页：请求分页是一种内存管理技术，它允许程序在需要时从磁盘或其他辅助存储中加载页面到物理内存中。

有效位 (装入位)：有效位通常用于标识缓存中的页面是否有效或已被加载。当页面在缓存中时，其有效位会被设置为 1；当页面不在缓存中或已被替换时，有效位会被设置为 0。

特权指令：特权指令是只能由操作系统或具有特权级别的程序执行的指令。这些指令通常用于执行系统级操作，如访问硬件、修改系统状态等。

存储保护：存储保护是操作系统提供的一种机制，用于防止程序访问或修改其不应该访问的内存区域，从而确保系统的稳定性和安全性。

物理页号：物理页号是物理地址中的一部分，用于标识物理内存中的页面。

未分配页：未分配页是尚未被进程申请使用的虚拟内存页面，它不占用物理内存或磁盘空间。

页故障 (Page Fault)：页故障是指在程序执行过程中，当试图访问一个尚未加载到物理内存中的虚拟页面时发生的情况。此时，系统需要执行页面替换和加载操作来解决故障。

页表：页表是操作系统用于记录虚拟内存页面与物理内存页面之间映射关系的数据结构。通过页表，CPU 可以将虚拟地址转换为物理地址。

修改位 (脏位)：修改位用于标识缓存中的页面是否被修改过。如果页面被修改过，其修改位会被设置为 1；否则，修改位为 0。这个信息在决定是否需要将页面写回到主存时非常重要。

特权模式：特权模式是操作系统的一种运行模式，在这种模式下，程序可以执行特权指令并直接访问系统资源。

地址越界：地址越界是指程序尝试访问其权限范围之外的内存地址。这通常会导致程序崩溃或系统错误。

访问权限：访问权限定义了程序或用户对特定内存区域的访问级别，如可读、可写或可

执行。

内核态：内核态是操作系统内核的运行模式，这种模式具有最高的权限级别，可以执行所有指令并访问所有内存区域。

访问越权：访问越权是指程序或用户尝试执行超出其权限范围的操作，如访问受保护的内存区域或执行特权指令。

二、简答题

(1)计算机内部为何要采用层次结构存储体系?层次结构存储体系如何构成?

解析：计算机内部采用层次结构存储体系的原因主要有两点：首先，解决快速的 CPU 和慢速的主存之间的速度差异；其次，解决主存容量不够大的问题。

层次结构存储体系由多个存储设备按照逻辑关系排列组合而成，主要包括缓存层、磁盘层、磁带库层和云存储层。缓存层最快，主要由高速缓存、RAM 等组成，用于加快数据访问速度；磁盘层由硬盘、SSD 等组成，用于存储数据；磁带库层主要用于存储备份数据；云存储层则通过云计算技术存储大量数据。

(2)为什么采用地址对齐方式能减少访问 DRAM 中数据的时间?

解析：采用地址对齐方式能减少访问 DRAM 中数据的时间，因为 DRAM 的访问是以块为单位的，如果数据跨越多个块，就需要进行多次访问。地址对齐可以确保数据按照块的大小来组织，从而在一次访问中获取所需的所有数据，减少访问次数，提高访问效率。

(3)为什么在 CPU 和主存之间引入 cache 能提高 CPU 的访存效率?

解析：在 CPU 和主存之间引入 cache 能提高 CPU 的访存效率，主要是因为 cache 具有存取速度快的特点，能够快速响应 CPU 的数据请求。当 CPU 需要访问主存中的数据时，可以先在 cache 中查找，如果找到，则直接从 cache 中读取数据，避免了对慢速主存的访问，从而提高了访存效率。

(4)为什么直接映射方式不需要考虑替换策略?

解析：直接映射方式不需要考虑替换策略，因为在这种映射方式下，每个主存页面直接映射到缓存中的特定页面，没有固定的对应关系。当缓存满时，新的页面会覆盖固定的对应页面，无需进行复杂的替换策略选择。

(5)为什么要考虑 cache 的写策略问题?

解析：需要考虑 cache 的写策略问题，是因为 CPU 对主存的访问与修改过程中，会引发 cache 内容的变化。如果不对 cache 和主存的所要修改的内容加以控制，两者的内容会变得不一致。为了确保数据的一致性，需要采用适当的写策略，如写回法或写通法，来决定何时以及如何将 cache 中的数据写回到主存中。

(6)什么是物理地址?什么是逻辑地址?地址转换由硬件还是软件实现?为什么?

解析：物理地址是内存中各存储单元的编号，即存储单元的真实地址，是可识别、可寻址并实际存在的。逻辑地址则是用户程序经过编译或汇编形成的目标代码中的地址，通常采

用相对地址形式。地址转换通常由硬件实现，如 CPU 中的地址映射机制，这是因为硬件转换速度更快，能够满足 CPU 对快速访问的需求。软件也可以实现地址转换，但速度相对较慢，一般用于更复杂的地址管理任务。

(7)在存储器层次化结构中,“cache-主存”“主存-外存”这两个层次有哪些不同?

解析：在存储器层次化结构中,“cache-主存”和“主存-外存”这两个层次的主要区别在于速度、容量和成本。cache-主存层次主要解决主存速度不快的问题，通过引入高速缓存来提高访问速度。主存-外存层次则主要解决主存容量不足的问题，通过外存设备来扩展存储容量。此外，这两个层次在访问频率和数据重要性方面也存在差异，cache 中的数据通常是频繁访问的热点数据，而外存中的数据则可能是长期保存但访问频率较低的数据。

一、名词解释

I/O 硬件：指的是输入/输出设备，如键盘、鼠标、显示器、磁盘等，这些硬件用于与计算机进行数据传输。

I/O 软件：是一组程序和工具，用于管理和控制 I/O 硬件的操作，包括设备驱动程序、中断处理程序、缓冲管理以及相关的系统调用等。

系统调用处理程序：是操作系统内核中的一部分，负责处理来自用户空间的系统调用请求，实现用户程序与内核之间的交互。

系统调用服务例程：是系统调用处理程序调用的内核函数，用于实现具体的系统调用功能。

系统级 I/O 函数：是操作系统提供的用于执行 I/O 操作的 API（应用程序接口），如 C 语言中的 stdio 函数库。

虚拟文件系统（VFS）：是一种用于网络环境的分布式文件系统接口，它允许不同的文件系统与操作系统进行交互，而无需考虑具体的文件系统实现细节。

流缓冲区：用于提高 I/O 操作的效率和性能，通常用于在数据读取或写入时进行暂存，以减少系统调用的次数和内存访问的延迟。

索引节点：通常用于文件系统中，包含文件的元数据（如文件大小、权限、创建时间等），是文件系统用来定位和访问文件的关键数据结构。

系统打开文件表：是操作系统内核维护的一个数据结构，用于记录当前打开的文件及其相关信息，如文件描述符、文件状态等。

磁盘高速缓存：是操作系统用于提高磁盘 I/O 性能的一种技术，通过缓存最近访问过的磁盘数据块，减少对磁盘的物理访问次数。

程序直接控制 I/O：指用户程序直接通过特定的 I/O 指令或系统调用进行 I/O 操作，而不依赖于操作系统提供的 I/O 软件。

就绪状态：在多任务操作系统中，通常用于描述一个进程或线程已经准备好执行，等待被调度器分配到 CPU 上运行的状态。

多重中断：指的是在一个中断处理程序执行过程中，发生了另一个中断，需要暂时保存当前中断的上下文并处理新中断的情况。

中断嵌套：与多重中断类似，但更强调中断之间的层次关系，即一个中断可以在另一个中断的处理过程中被触发和处理。

设备控制器：是硬件设备与计算机之间的接口电路，负责控制和管理硬件设备的操作，如磁盘控制器、显卡控制器等。

I/O 端口：是 CPU 与外部设备通信的接口，用于发送和接收数据以及控制信号。

状态端口：是 I/O 端口的一种，用于读取或设置硬件设备的状态信息。

I/O 地址空间：是计算机内存中用于映射 I/O 端口和设备控制器的地址范围，使得 CPU 可以通过内存访问的方式与 I/O 设备进行通信。

存储器映射 I/O：是一种 I/O 技术，通过将 I/O 端口或设备控制器映射到内存地址空间，使得 CPU 可以像访问内存一样访问这些 I/O 设备。

I/O 指令：是 CPU 执行的指令，用于启动 I/O 操作，如读取或写入数据。

中断响应优先级：是操作系统用来决定哪个中断应该首先被响应的优先级规则。

中断处理优先级：是操作系统用来决定哪个中断处理程序应该首先被执行的优先级规则。

用户空间 I/O 软件：用户空间 I/O 软件是运行在操作系统用户空间中的程序，用于处理输入/输出 (I/O) 操作。这些软件通常通过系统调用来与内核空间 I/O 软件进行交互，从而实现了对硬件设备的访问和控制。

内核空间 I/O 软件：内核空间 I/O 软件是操作系统内核中负责处理 I/O 操作的部分。它管理硬件设备的驱动程序，处理中断，以及提供系统调用接口供用户空间程序使用。

设备驱动程序：设备驱动程序是一种软件，用于控制计算机上的硬件设备（如打印机、键盘、鼠标、音频/视频设备等），以便它们能够与计算机系统协调工作。设备驱动程序通常由设备制造商提供，并与操作系统紧密集成，以便系统可以识别和使用设备。

文件描述符：文件描述符是由无符号整数表示的句柄，进程使用它来标识打开的文件。文件描述符与包括相关信息（如文件的打开模式、文件的位置类型、文件的初始类型等）的文件对象相关联。

目录文件：目录文件是文件系统中的特殊文件，用于存储多个文件目录的信息。它包含了多个文件或目录的元数据，使得文件系统能够方便地管理和查找这些文件或目录。

高速缓存 RAM：高速缓存 RAM (Random Access Memory) 是一种用于暂时存储数据和程序的内存设备。它具有快速的读写能力，并通常用于存储 CPU 经常访问的数据，以提高数据访问速度。

中断控制 I/O：中断控制 I/O 是一种 I/O 控制方式，其中当外部设备准备好进行数据传输时，它会向 CPU 发送一个中断信号。CPU 在接收到中断信号后，会暂停当前的任务，转而执行中断处理程序来处理 I/O 操作。

DMA 方式：DMA (Direct Memory Access) 方式是一种允许某些硬件子系统（如磁盘控制器）直接访问主存，而无需通过 CPU 进行数据传输的技术。它提高了数据传输的效率，减少了 CPU 的介入。

控制端口：控制端口是计算机系统中用于发送和接收控制信号的端口。它通常用于与硬件设备通信，以控制设备的操作和行为。

独立编址方式：独立编址方式是指 I/O 端口地址和内存地址相互独立，使用不同的地址空间进行编址。这种编址方式使得 I/O 端口和内存可以互不干扰地进行访问。

中断服务程序：中断服务程序是操作系统内核中用于处理特定中断事件的程序。当中断发生时，CPU 会跳转到相应的中断服务程序执行，以处理中断事件。

文件元数据：文件元数据是关于文件自身的信息，如文件的创建时间、修改时间、大小、权限等。这些信息通常存储在文件系统的元数据结构中，用于管理和访问文件。

目录项：目录项是文件系统中用于记录文件或目录信息的条目。它包含了文件或目录的名称、属性以及其他相关元数据，使得用户可以方便地查找和访问这些文件或目录。

I/O 控制方式：I/O 控制方式是指计算机系统中用于管理 I/O 操作的方法和技术。常见的 I/O 控制方式包括程序直接控制、中断驱动、DMA 等。

中断屏蔽字：中断屏蔽字是一种用于控制中断响应的寄存器或变量。通过设置中断屏蔽字，可以屏蔽或允许特定的中断请求，以实现中断的优先级管理。

DMA 控制器：DMA 控制器是一种硬件设备，用于实现 DMA 方式的数据传输。它负责在内存和设备之间直接传输数据，而无需 CPU 的干预。

数据端口：数据端口是计算机系统中用于传输数据的端口。它通常用于与硬件设备通信，以发送和接收数据。

统一编址方式：统一编址方式是指将 I/O 端口地址和内存地址统一进行编址，使用相同的地址空间。这种编址方式简化了 I/O 操作的访问方式，但也可能导致内存空间的浪费。

可编程中断控制器：可编程中断控制器是一种用于管理中断请求的硬件设备。它允许操作系统对中断进行优先级排序、屏蔽和响应控制，以实现中断的高效处理。

中断请求寄存器：中断请求寄存器是用于记录当前待处理的中断请求的寄存器。当中断发生时，相应的中断请求会被记录到中断请求寄存器中，以供 CPU 或中断控制器处理。

二、简答题

(1) I/O 子系统的层次结构是怎样的？

解析：I/O 子系统的层次结构通常从上至下依次为：用户层 I/O 软件、设备独立性软件、设备驱动软件、中断处理软件和硬件（包括机械部件和电子部件）。这种层次结构的设计使得 I/O 操作更加模块化和抽象化，方便了软件的开发和维护。

(2) 系统调用封装函数对应的机器级代码结构是怎样的？

解析：系统调用封装函数对应的机器级代码结构通常包括若干条传送指令后跟一条陷阱指令。传送指令用于传递系统调用的参数，而陷阱指令（如 `int $0x80`）则用于使 CPU 陷入内核进行处理。这种结构是系统调用在机器层面的实现方式。

(3) 为什么系统调用的开销很大？

解析：系统调用的开销很大，主要是因为系统调用涉及从用户态切换到内核态的过程，这需要时间进行上下文切换。此外，在内核态执行时，CPU 工作在管理员模式，这涉及到特殊的堆栈和内存环境，需要在系统调用开始时建立并在结束后恢复。这种运行环境的切换以及内核态下对资源的访问控制都增加了系统调用的开销。

(4) C 标准 I/O 库函数是在用户态执行还是在内核态执行？

解析：C 标准 I/O 库函数（如 `printf` 和 `scanf`）是在用户态执行的。这些函数提供了用户程序与 I/O 设备之间的接口，但实际的 I/O 操作（如读写数据）是由内核在内核态下完成的。用户态的程序通过系统调用请求内核进行这些操作。

(5) 与 I/O 操作相关的系统调用封装函数是在用户态执行还是内核态执行？

解析：与 I/O 操作相关的系统调用封装函数是在内核态执行的。当用户程序需要执行 I/O 操作时，它会通过系统调用接口请求内核执行相应的 I/O 操作。内核会切换到内核态，执行相应的系统调用封装函数，完成 I/O 操作后再返回到用户态。

(6) 什么是程序直接控制 I/O 方式？说明其工作原理。

解析：程序直接控制 I/O 方式是指 CPU 直接通过 I/O 指令对 I/O 设备进行操作的方式。在这种方式下，CPU 会向 I/O 设备发送控制命令和数据，然后等待设备完成操作并返回状态信息。这种方式的缺点是 CPU 需要频繁地检查设备的状态，降低了 CPU 的效率。

(7)为什么在保护现场和恢复现场的过程中，CPU 必须关中断？

解析：在保护现场和恢复现场的过程中，CPU 必须关中断，这是因为中断会打断当前的执行流程，如果允许中断发生，可能会干扰正在处理中的操作系统的信息，导致恢复现场时状态不一致，甚至发生错误。关中断可以确保在恢复现场之前，不会有其他中断打断当前的处理过程。

(8)什么是中断控制 I/O 方式?说明其工作原理。

解析：中断控制 I/O 方式是一种通过中断机制进行 I/O 操作的方式。当 I/O 设备准备好进行数据传输时，它会向 CPU 发送一个中断信号。CPU 在接收到中断信号后，会暂停当前的任务，转而执行中断处理程序来处理 I/O 操作。这种方式可以实现 CPU 与 I/O 设备之间的并行操作，提高了系统的效率。

(9)DMA 方式能够提高成批数据交换效率的主要原因何在？

解析：DMA 方式能够提高成批数据交换效率的主要原因在于它允许数据直接在内存和设备之间进行传输，而无需 CPU 的介入。这种方式减少了 CPU 的开销，使得 CPU 可以专注于执行其他任务。同时，DMA 方式还可以实现高速的数据传输，进一步提高了数据交换的效率。

(10)DMA 控制器在什么情况下发出中断请求信号？

解析：DMA 控制器在数据块传输完毕时发出中断请求信号。这是因为 DMA 控制器负责在内存和设备之间直接传输数据块，当一块数据传输完成后，它需要通知 CPU 以便 CPU 可以进行后续的处理或启动新的数据传输任务。通过发出中断请求信号，DMA 控制器可以确保 CPU 能够及时响应并处理数据传输的完成情况。

(11) I/O 端口的编址方式有哪两种?各有何特点？

解析：I/O 端口的编址方式主要有独立编址和统一编址两种。独立编址方式下，I/O 端口地址和内存地址相互独立，使用不同的地址空间进行编址。这种方式使得 I/O 端口和内存可以互不干扰地进行访问。统一编址方式下，I/O 端口地址和内存地址统一进行编址，使用相同的地址空间。这种方式简化了 I/O 操作的访问方式，但可能导致内存空间的浪费。

(12)为什么中断控制器把中断类型号放 I/O 总线的数据线上而不是放在地址线上？

解析：中断控制器把中断类型号放在 I/O 总线的数据线上而不是放在地址线上的原因主要是为了适应中断处理机制。中断类型号标识了发生中断的具体原因或来源，它需要通过 I/O 总线传递给 CPU。将数据线用于传输中断类型号可以确保数据在传输过程中的准确性和可靠性。而地址线主要用于在内存或 I/O 设备中定位特定的存储单元或端口，其传输的数据与中断处理不直接相关。因此，将中断类型号放在数据线上是更合理的选择。